KNOWALL

Module 4 — Locality Intelligence & Community Discovery

Technical report: features, design, implementation, screens and verification

Project	KNOWALL — residential community platform (Green Valley Residency, fictional)
Module	4 of 6 — Locality Intelligence & Community Discovery
Stack	Flask 3 · SQLAlchemy 2 · Alembic · PostgreSQL 18 · Next.js 16 (App Router) · TypeScript
Migration	e5c1a9d3f742 (additive; revises d9a4f2c6b871)
Report date	29 October 2026
Verification	Backend 250/250 tests passed · Frontend 92/92 tests passed · typecheck, lint and production build clean · 15-step HTTP end-to-end demo passed

Every place, phone number, event and alert in this report is **fictional demonstration data**, labelled "(demo)" in the application. Phone numbers use the 00000 pattern and are not real.

Contents

1. Executive summary **3**

At a glance . . . 3

2. Features implemented **4**

3. Architecture **5**

4. Roles and permissions **6**

5. Data model **7**

6. Essential services directory **8**

7. Resident corrections and moderation **10**

Issue types and what they change . . . 10

Conflict protection . . . 10

8. Local events, alerts, feed and reports **12**

9. Frontend walkthrough **13**

Resident . . . 14

Society Admin . . . 19

Locality Coordinator . . . 21

After approval . . . 28

10. API reference **31**

11. Validation, security and privacy **33**

12. Testing and verification **34**

What the backend tests cover . . . 34

13. Seed data and demo accounts **35**

14. How to run Module 4 **36**

Demo script (summary) . . . 36

15. Limitations and future work **37**

16. Files added and changed **37**

1. Executive summary

Module 4 gives residents of Green Valley Residency a trustworthy view of the neighbourhood around them: a directory of essential services (pharmacies, clinics, police, emergency and public services) where every listing shows **where its information came from** and **when it was last verified**, plus moderated local events, alerts and a curated feed.

Because directories go out of date, residents can report outdated information. A dedicated **Locality Coordinator** reviews each report, compares what was listed when the resident reported with what is published now, and approves, rejects or asks for clarification — always with a recorded reason. Every change is versioned, written to an append-only history and mirrored to the platform audit log.

Main demo story: Rahul (resident) finds Meadow Care Pharmacy, notices its phone number is outdated and reports it. Kavya (Locality Coordinator) reviews the report and approves it with a reason. The listing updates, the feed shows "Listing corrected", and Rahul sees the outcome and reason in *My corrections* — without seeing who reviewed it.

At a glance

Item	Delivered
Database	9 new tables, 1 additive Alembic migration, role column widened for the new role
Backend API	37 operations on 34 paths in one Flask blueprint (`locality_bp`), documented in OpenAPI
Frontend	13 new pages: 6 resident pages, 7 coordinator desk pages; locality sections in the sidebar
Access control	New role LOCALITY_COORDINATOR; new permissions locality.read and locality.manage; Society Admin deliberately excluded from management
Seed data	1 fictional locality, 6 categories, 16 places in every verification state, 4 events, 3 alerts, 1 coordinator account — repeat-safe
Tests	15 new backend API tests and 12 new frontend tests; full suites: 250 backend and 92 frontend, all passing
Docs	docs/module-4/README.md (setup, rules, demo script, limitations), OpenAPI contract, this report

2. Features implemented

The table lists every feature in the Module 4 specification, its status and where it lives. Paths are relative to the repository root; backend files are under `backend/app/modules/locality/`.

Feature	Status	Where / how
Locality profile with radius coverage and configurable re-verification window	Implemented	directory.py · /locality · desk settings
Coverage check contract for Module 5	Implemented	GET /api/locality/coverage/check (haversine distance)
Essential services directory: categories, server-side search, filters, pagination	Implemented	directory.py · /locality/places
Map discovery with list ↔ map selection sync	Implemented (schematic SVG)	CoverageMap in features/locality/shared.tsx
Place detail: source, last-verified date and method, stale indicator, history	Implemented	/locality/places/[id]
Local events: resident submission, coordinator approval, upcoming/past, categories	Implemented	content.py · /locality/events · desk Events
Module 3 society events shared only via opt-in + Society Admin share + coordinator approval; RSVPs never exposed	Implemented	POST /api/locality/events/submit with source_event_id
Moderated alerts: severity, source attribution, draft→publish, expiry, archive	Implemented	content.py · /locality/alerts · desk Alerts
Curated feed: pinned active alerts, events, directory updates	Implemented	GET /api/locality/feed · /locality
Content reports on events and alerts	Implemented	POST /api/locality/reports · desk Reports
Coordinator stewardship: create, edit (versioned), verify, flag for recheck, archive, restore, history	Implemented	/locality/manage/places and editor
Resident corrections with mandatory reason, clarification round-trip, My corrections	Implemented	corrections.py · /locality/corrections
Conflict protection (version check, value snapshot, row locks, duplicate & rate limits)	Implemented	directory.py · corrections.py
Missing-service reports	Implemented	POST /api/locality/corrections
History + audit for every change	Implemented	locality_history + audit_logs (LOCALITY_* actions)
Fictional, clearly labelled seed data (≥ 15 records) and demo accounts	Implemented	backend/scripts/seed_locality.py
Notifications to reporters	Not implemented	No platform notification contract yet (same as Modules 2–3)
Real map tiles / geocoding	Not implemented by design	See limitations (section 15)

3. Architecture

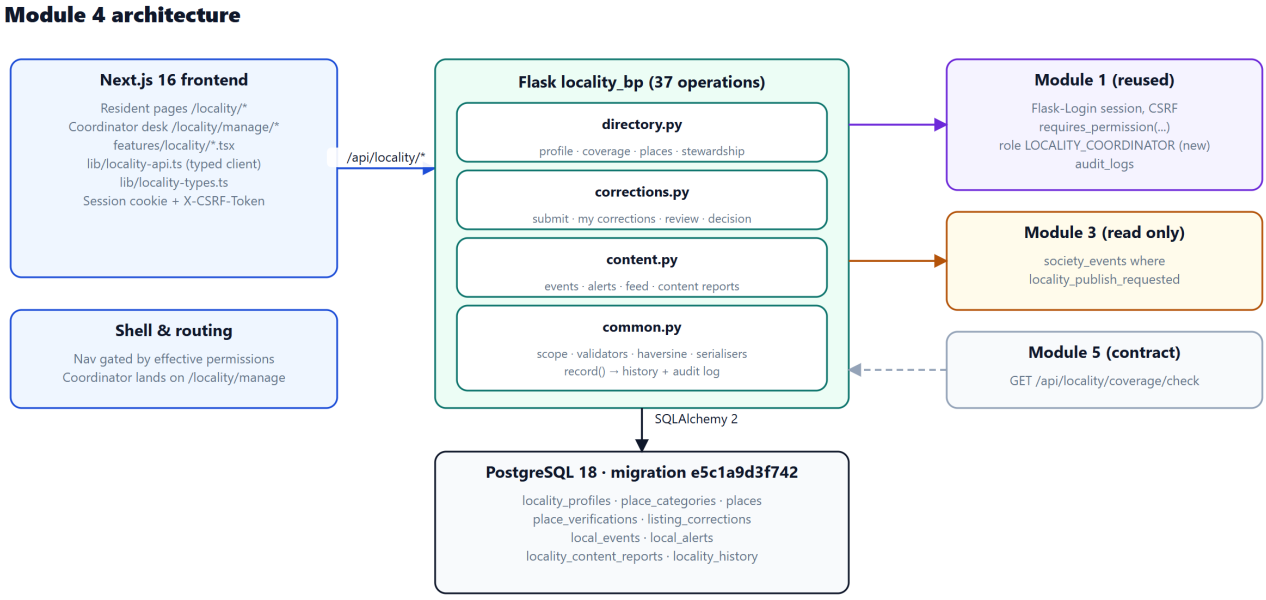


Figure 1 — Module 4 sits beside Modules 1–3 and reuses Module 1's session, CSRF, permission and audit infrastructure.

Backend. One Flask blueprint split into four files. `common.py` resolves the locality from the configured society (the client never sends a locality id), validates input (text lengths, http(s)-only URLs, phone format, coordinates, time zones, enums), computes distances with the haversine formula, serialises records and writes history. `directory.py` serves the profile, places and stewardship; `corrections.py` runs the correction workflow; `content.py` handles events, alerts, the feed and reports.

Frontend. Typed API client (`lib/locality-api.ts`) and types (`lib/locality-types.ts`); feature components in `features/locality/` and `features/locality/manage/`; thin route files under `app/locality/`. Every screen checks permissions for display, and Flask enforces them again on every request.

Integration points. Module 1: sessions, CSRF header, `requires_permission`, roles and `audit_logs`. Module 3: read-only use of `society_events` that opted in. Module 5: a documented coverage-check endpoint and stable category keys.

4. Roles and permissions

Module 4 reuses Module 1's permission system. Two permissions and one role were added; nothing existing was removed or renamed.

Permission	Who has it	Allows
locality.read (new)	Eligible residents, verified Society Admin, Locality Coordinator	Browse profile, places, events, alerts, feed
locality.listings.suggest (existing)	Eligible residents	Submit corrections, missing services, event suggestions, content reports; reply to clarifications
locality.manage (new)	LOCALITY_COORDINATOR only	All /api/locality/admin/* routes: stewardship, moderation, profile settings
society.operations.manage (Module 3)	Society Admin	List shareable society events and share them with the locality

Action	Applicant	Resident	Society Admin	Platform Admin	Coordinator
Browse directory, events, alerts	□ 403	□	□	□	□
Report a correction / suggest event	□	□	□	□	only if also resident
Share an opted-in society event	□	□	□	□	□
Review corrections, publish events/alerts	□	□	□ 403	□ 403	□ (not own reports)
Create / edit / verify / archive places	□	□	□ 403	□ 403	□

- **Why a separate role?** The specification requires that a Society Admin does not automatically receive locality permissions. Directory stewardship is a distinct duty, so it is granted deliberately through Module 1's existing `role_assignments` table.
- The coordinator role needs no household membership. A person who is both resident and coordinator can moderate, but Flask refuses a decision on their own correction (`403 SELF_REVIEW_FORBIDDEN`).
- A coordinator who is not a resident is routed to `/locality/manage` after login (`lib/login-routing.ts`).
- Sidebar groups are rendered from the effective permissions returned by Flask: LOCALITY for locality.read, LOCALITY DESK for locality.manage, and a small LOCALITY group in the admin workspace.

5. Data model

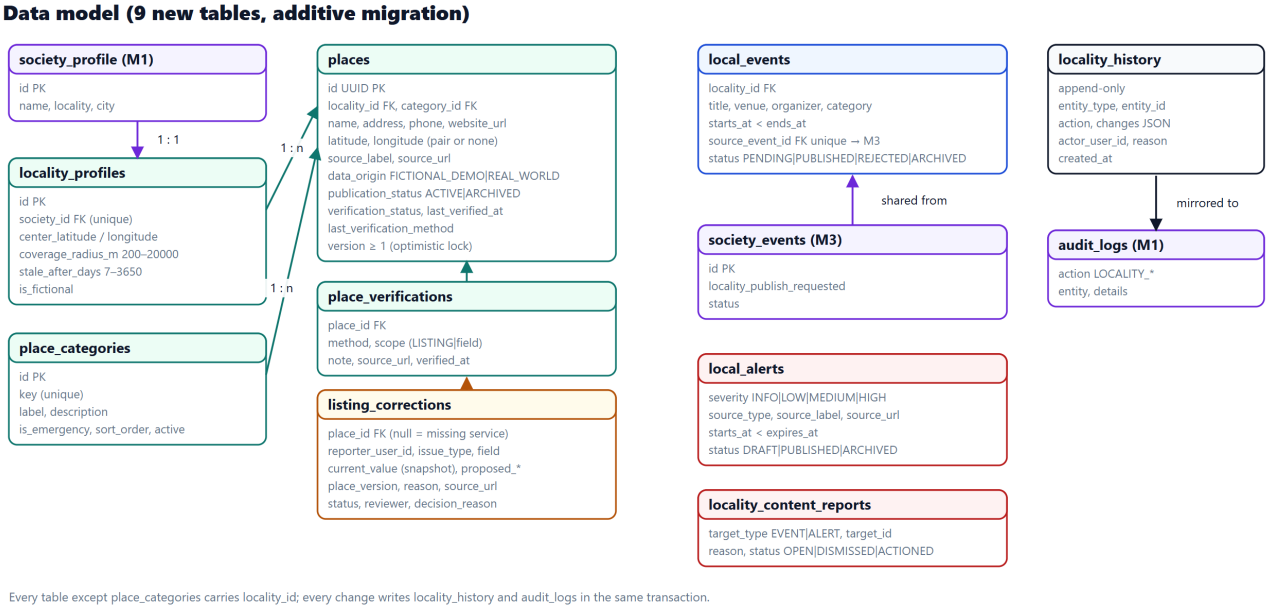


Figure 2 — Nine new tables. Every row is scoped to a locality, which belongs to the configured society.

Table	Purpose and key rules
locality_profiles	One per society. Centre point, coverage radius (200–20 000 m), stale_after_days (7–3650, default 180), is_fictional flag.
place_categories	Stable keys used by the UI and Module 5: pharmacy, hospital_clinic, police, emergency, public_service, other_service. is_emergency flag.
places	Directory listing. Coordinates must be both present or both absent and inside coverage. source_label is mandatory; REAL_WORLD needs source_url. VERIFIED requires a date. version ≥ 1 for optimistic locking.
place_verifications	Append-only verification log. scope = LISTING for a full check, or the single field a correction review confirmed.
listing_corrections	Resident reports. Stores a snapshot of the published value and the place version at submission, the proposal, reason, optional evidence link, status and decision reason.
local_events	Events with a time window check. source_event_id (unique) links a shared Module 3 event.
local_alerts	Severity, source type and label; OFFICIAL_NOTICE requires a source URL (database check). expires_at > starts_at.
locality_content_reports	Resident flags on published events and alerts, with review outcome.
locality_history	Append-only change log: entity, action, JSON diff {field: [old, new]}, actor, reason.

The migration is additive: it creates the nine tables with foreign keys, indexes and check constraints, and widens `role_assignments.role` from VARCHAR(16) to VARCHAR(30) because "LOCALITY_COORDINATOR" is 20 characters. A full downgrade is provided. `flask db check` reports no drift between models and the PostgreSQL schema.

6. Essential services directory

Residents search by name, address, description or category (server-side, case-insensitive, with % and _ escaped so they are not wildcards) and filter by category. Results are paginated. Archived listings are never returned to residents.

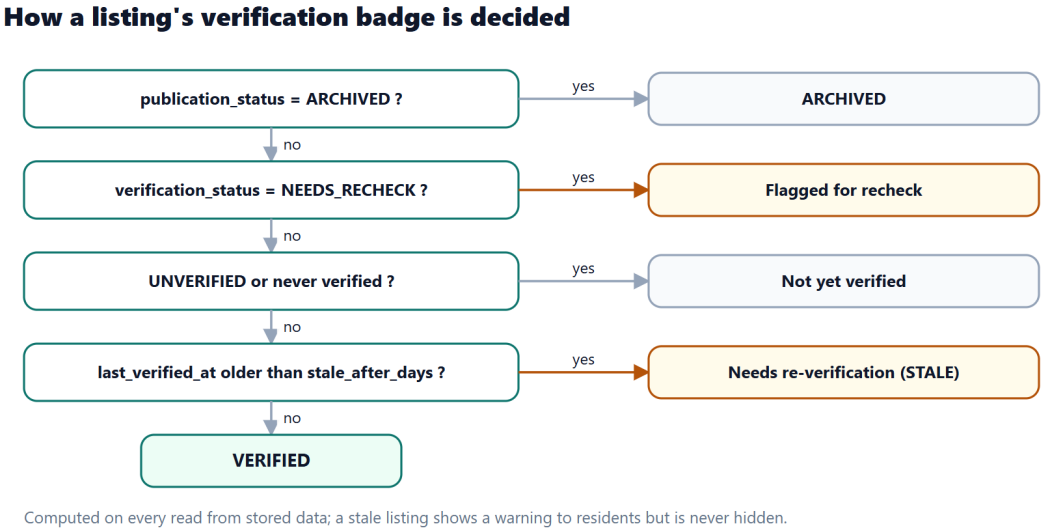


Figure 3 — The verification badge is computed on every read from stored data.

- **Stale indicator.** A listing verified longer ago than `stale_after_days` shows “Needs re-verification” and a warning to call ahead. It is never silently hidden.
- **Source attribution.** Every detail page shows the source label, optional source link, last-verified date and verification method, and whether the listing is fictional or real-world.
- **Emergency categories** show “In a real emergency, dial 112.”
- **Map.** A schematic SVG draws the coverage circle (with half-radius ring and compass) and plots each place by distance and direction from the society. Selecting a marker scrolls to its result and hovering a result highlights its marker; markers are keyboard-accessible buttons. Places without coordinates are counted under the map.

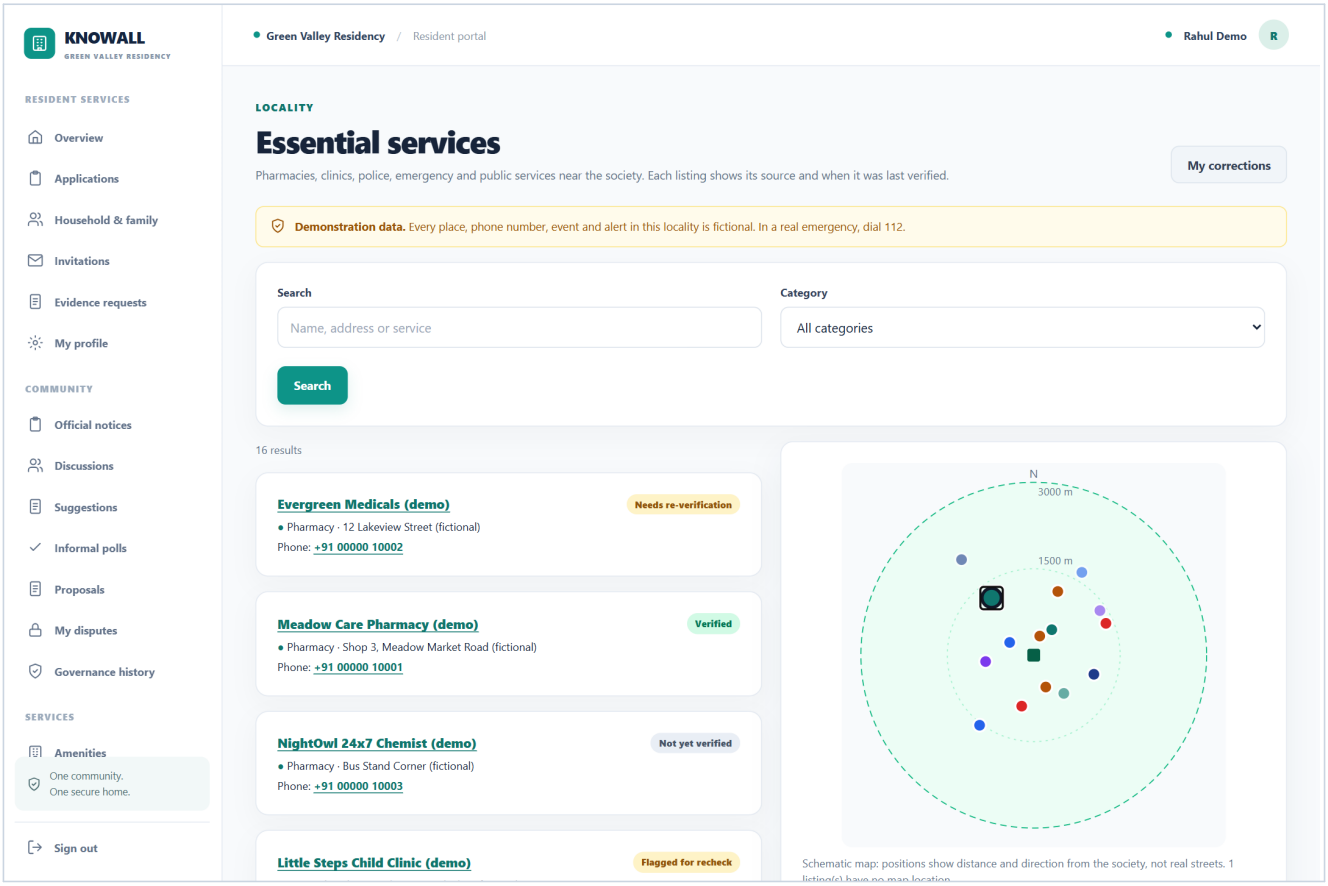


Figure 4 — Resident directory with fictional-data banner, filters, verification badges and the schematic coverage map (a marker is selected).

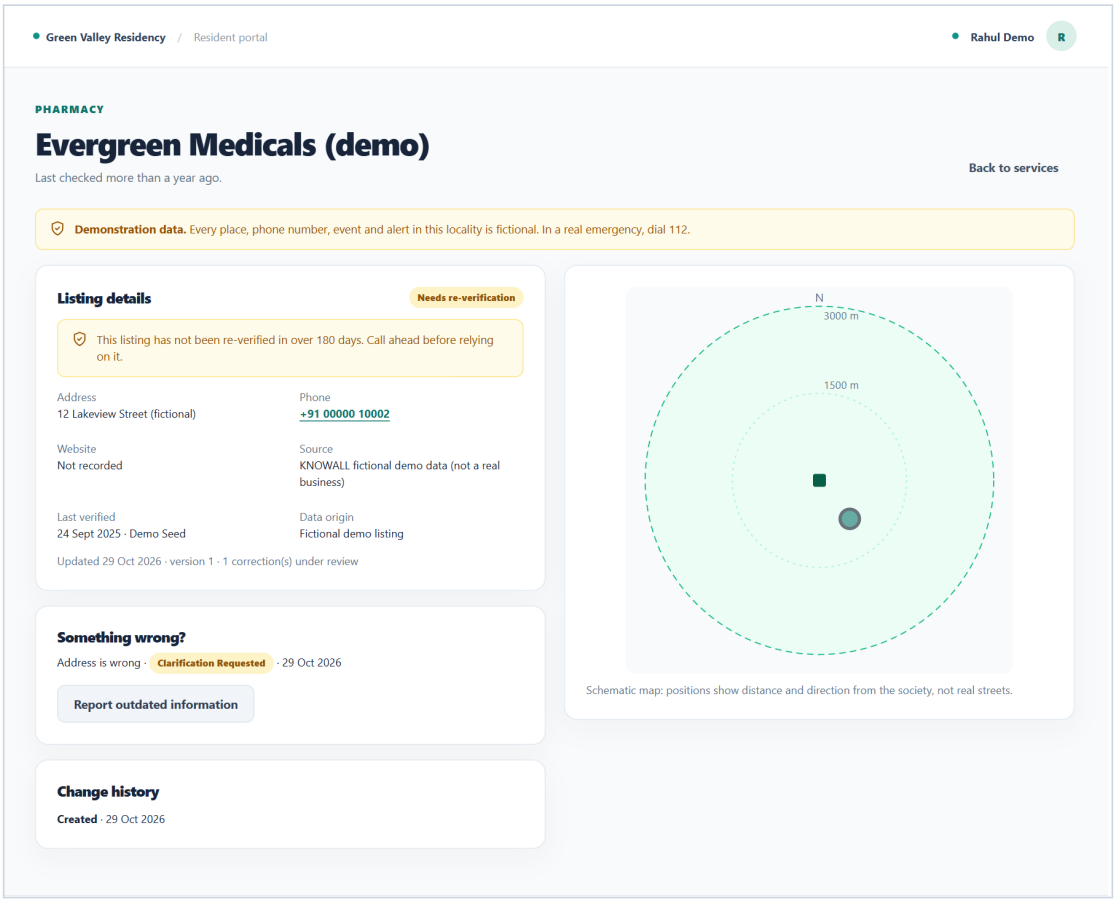


Figure 5 — A stale listing: the warning, source, last-verified date and method are always visible.

7. Resident corrections and moderation

This is the core workflow of the module and the main demo story.

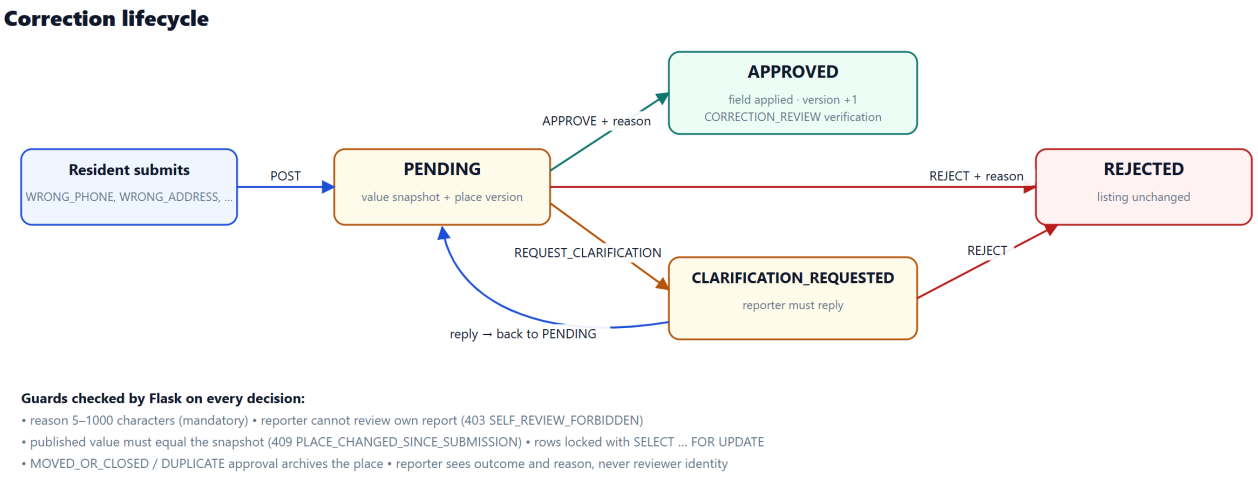


Figure 6 — Correction states. Every decision needs a reason; Flask checks the guards listed.

Issue types and what they change

Issue type	Field	Proposal required	On approval
WRONG_PHONE	phone	Valid phone format	Phone replaced
WRONG_ADDRESS / WRONG_NAME	address / name	Text (length limits)	Field replaced
WRONG_WEBSITE	website_url	http(s) link only	Field replaced
WRONG_COORDINATES	coordinates	Lat/lng inside coverage	Map point moved (re-checked against current coverage)
WRONG_CATEGORY	category	Existing category key	Category changed
MOVED_OR_CLOSED / DUPLICATE	—	Reason only	Place archived
MISSING_SERVICE	—	Name + category	Coordinator creates the listing
OTHER	—	Optional text	Recorded; coordinator edits manually

Conflict protection

- **Snapshot check.** When a report is submitted, the current published value and place version are stored. If the listing changes before review (for example the coordinator edited it directly), approval is refused with `409 PLACE_CHANGED_SINCE_SUBMISSION` and the UI shows “Listing changed since report”.
- **Row locks.** The correction and the place are locked (`SELECT … FOR UPDATE`) during a decision, so two coordinators cannot apply conflicting decisions at once.
- **Competing reports.** The queue shows how many other open reports target the same field, and the review panel lists them side by side.
- **Limits.** One open report per reporter per place and field (`409 DUPLICATE_OPEN_CORRECTION`); at most 10 open reports per resident (429). A proposal equal to the current value is rejected.
- **Privacy.** The reporter sees status, decision and reason, never the reviewer's identity. Other residents see only the public change history, without names.
- **Verification semantics.** Approving records a `CORRECTION_REVIEW` verification scoped to that single field — it does not mark the whole listing verified, so a stale listing stays stale until fully re-checked.

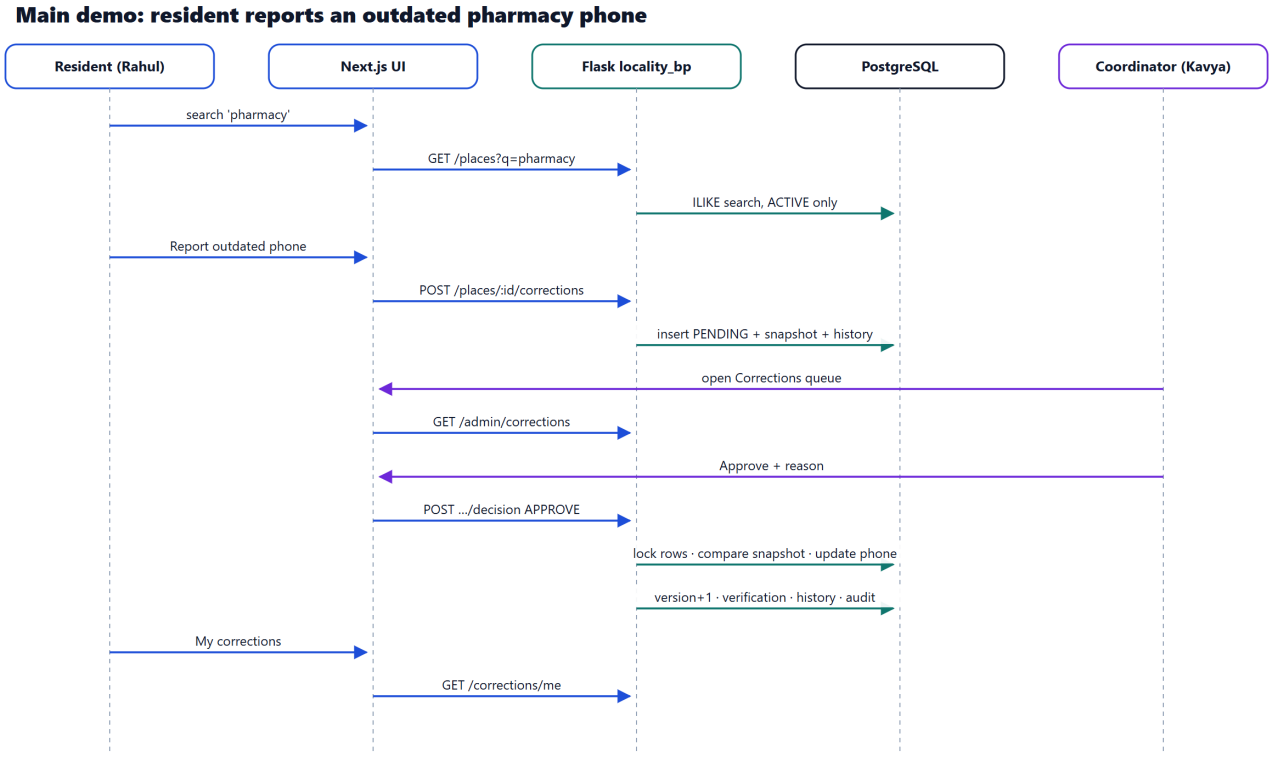


Figure 7 — Main demo as a sequence across the browser, Flask and the database.

8. Local events, alerts, feed and reports

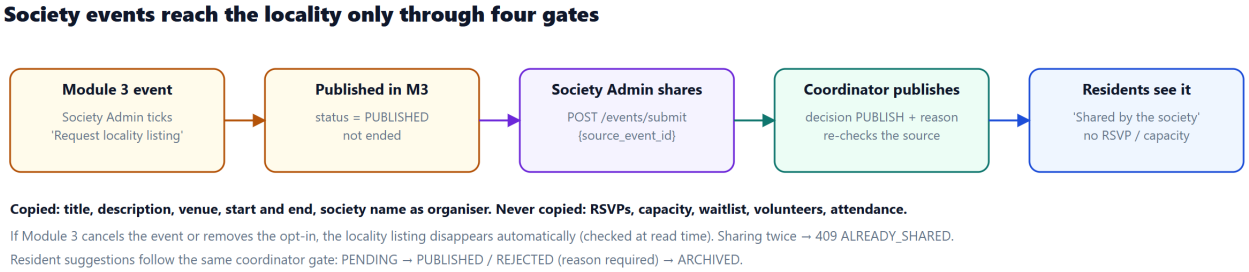


Figure 8 — A Module 3 society event reaches the locality only through four independent gates.

Residents can suggest local events (title, organiser, venue, category, start and end with a time zone; at most 14 days; not already ended). Suggestions start as PENDING and appear only after a coordinator publishes them. Events can be filtered by upcoming/ongoing or past, and by category.

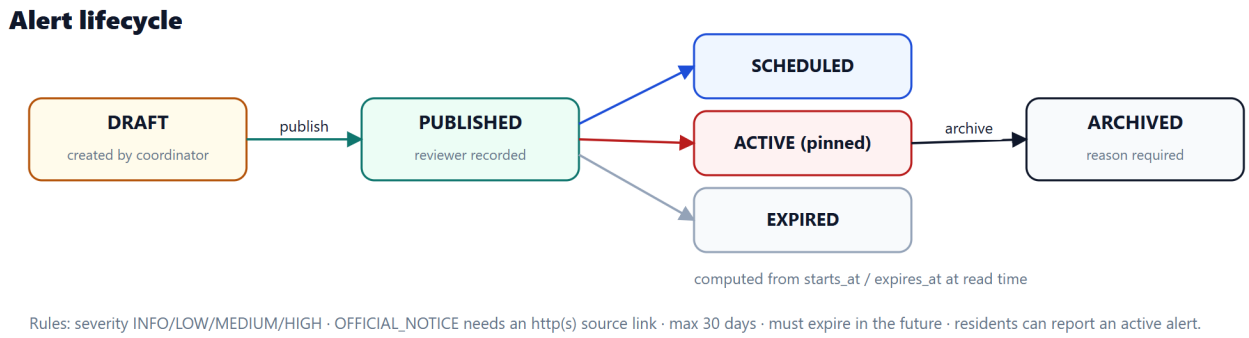


Figure 9 — Alerts are drafted, published with a recorded reviewer, expire automatically and can be archived with a reason.

- **Feed.** A read-through view (nothing is copied): active alerts pinned and sorted by severity, then upcoming events and recent directory updates (new listing, listing corrected, listing re-verified) from the last 30 days.
- **Content reports.** Residents can report a published event or alert with a reason. The coordinator either dismisses the report or archives the content, which settles every open report on the same item. Duplicate open reports are refused.

9. Frontend walkthrough

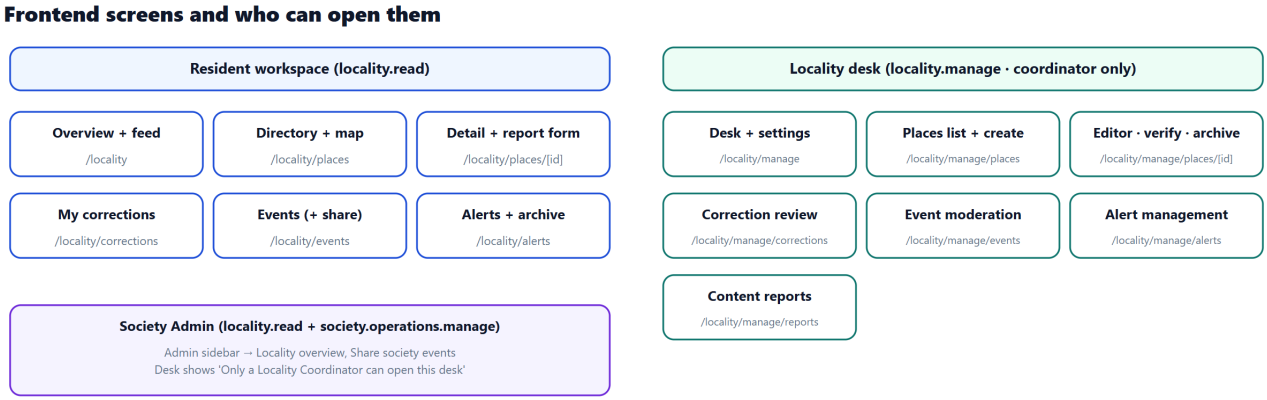


Figure 10 — Screens and the permission each requires.

The following screenshots were captured from the running Next.js application against a scratch copy of the seeded database, following the demo script. Sidebars are cropped except where navigation is the point.

Resident

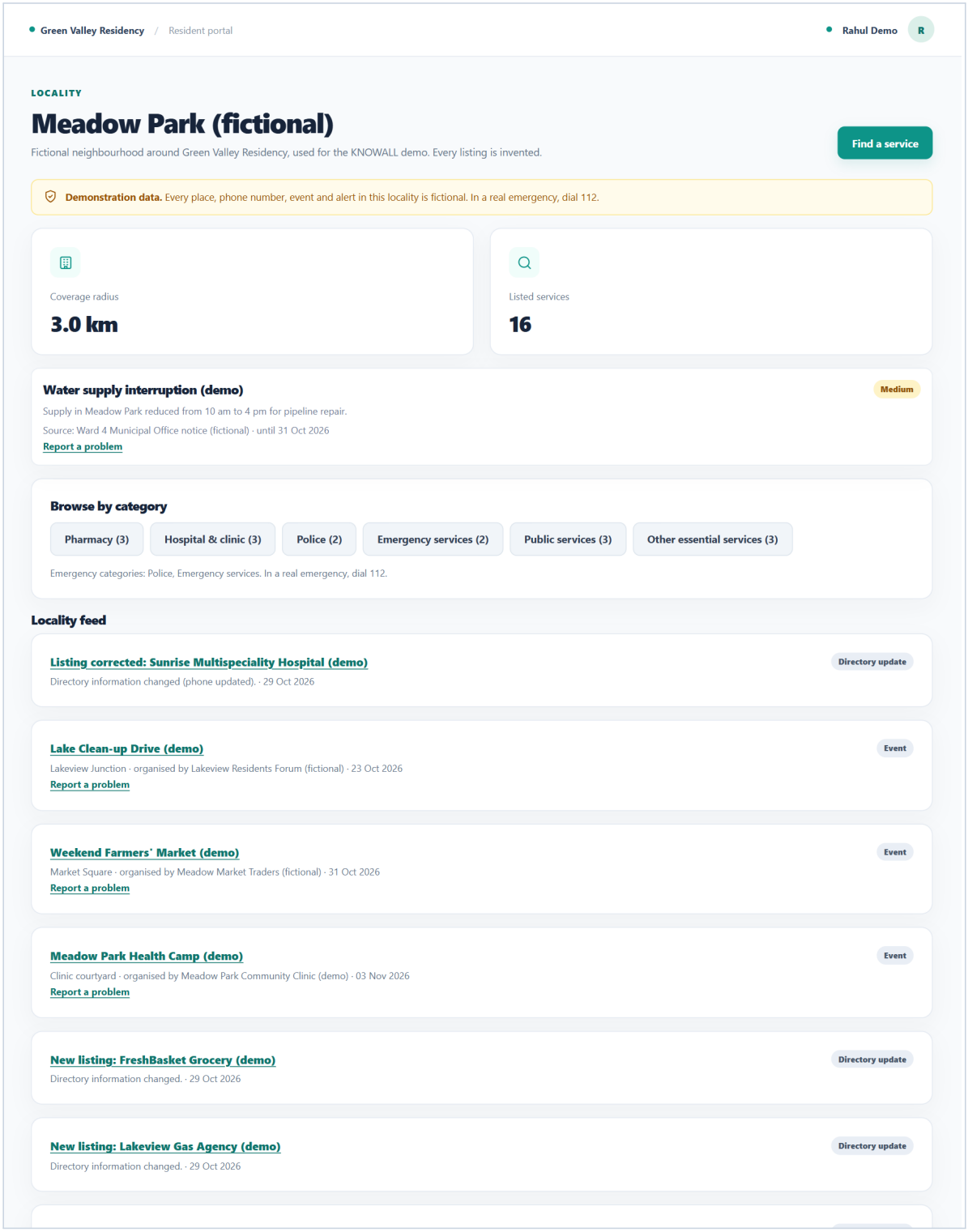


Figure 11 — Locality overview: fictional-data banner, coverage, active alert pinned, categories with counts, and the feed.

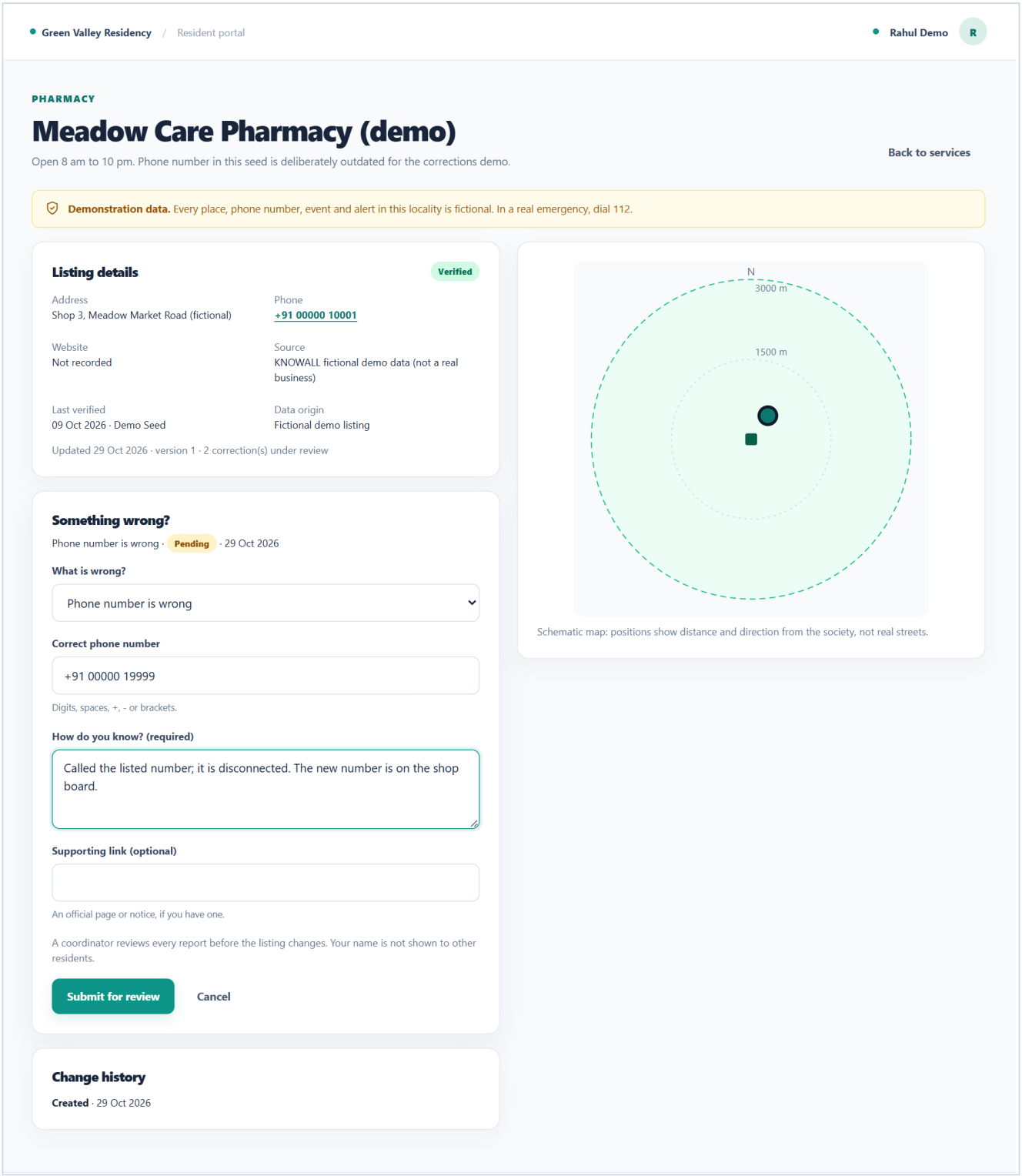


Figure 12 — Place detail for Meadow Care Pharmacy with the correction form open: issue type, proposed phone, mandatory reason, optional evidence link.

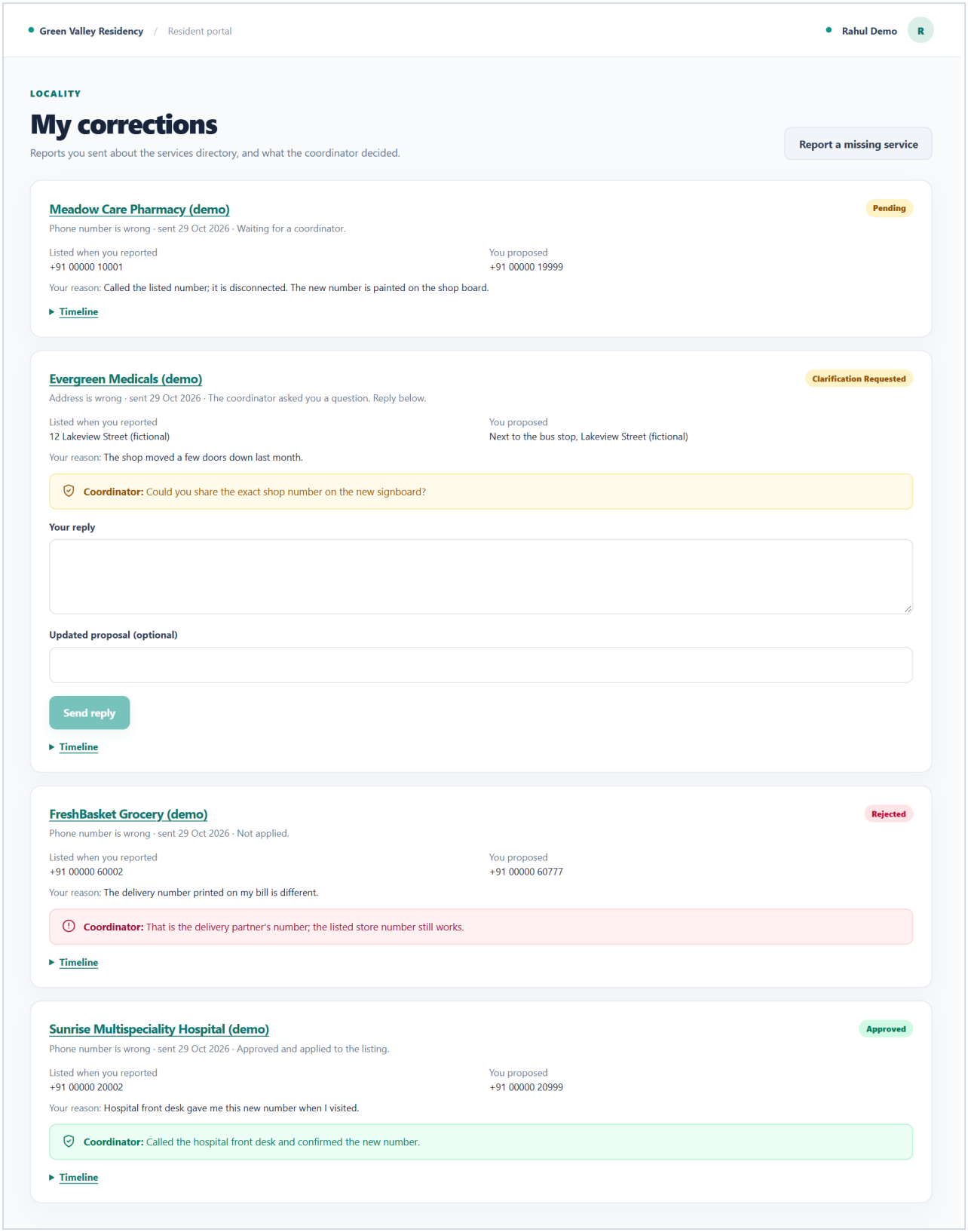


Figure 13 — My corrections before review: pending report plus earlier approved, rejected and clarification-requested outcomes with the coordinator's reasons.

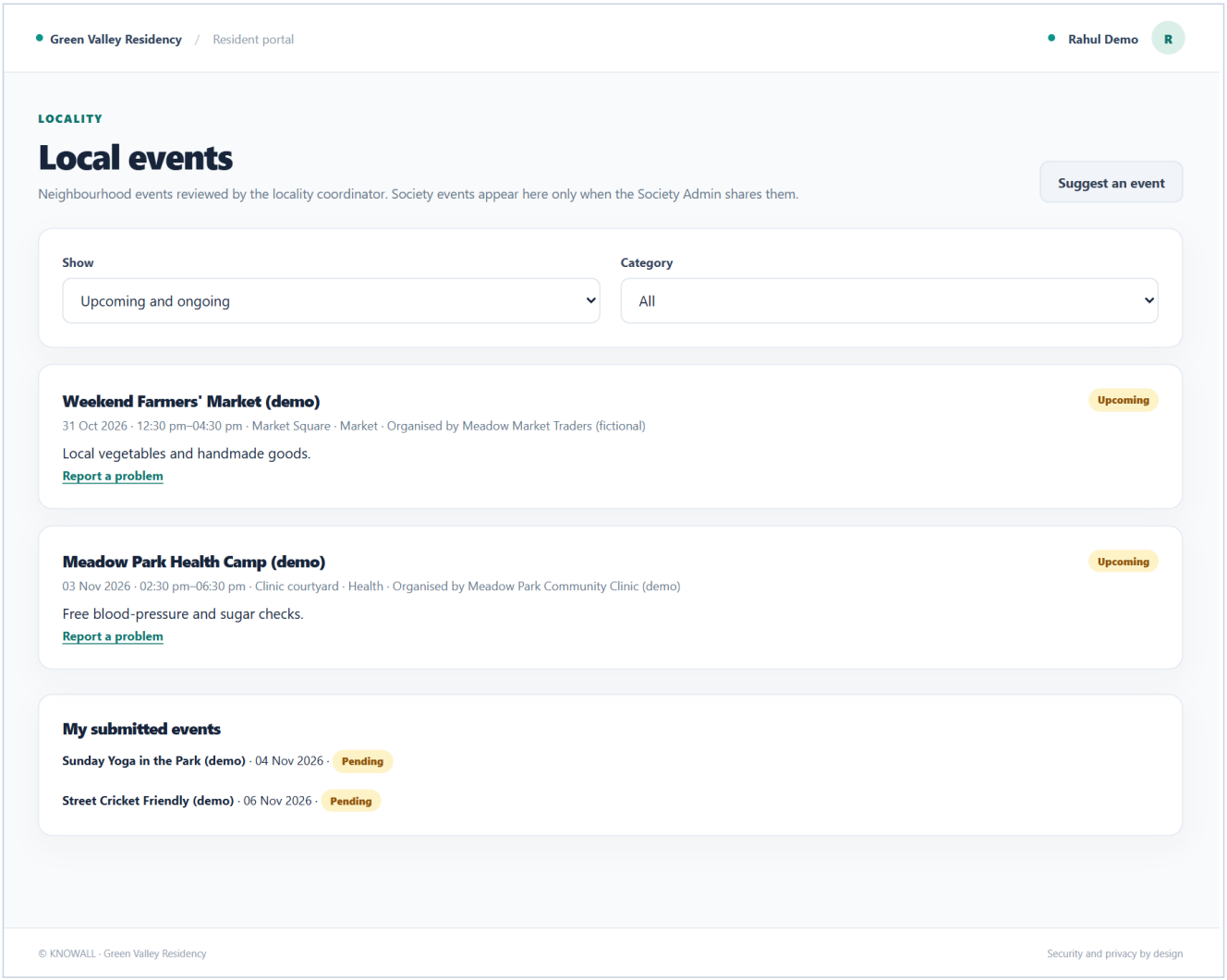


Figure 14 — Local events: published events, the resident's own suggestion with its status, and the suggestion form entry point.

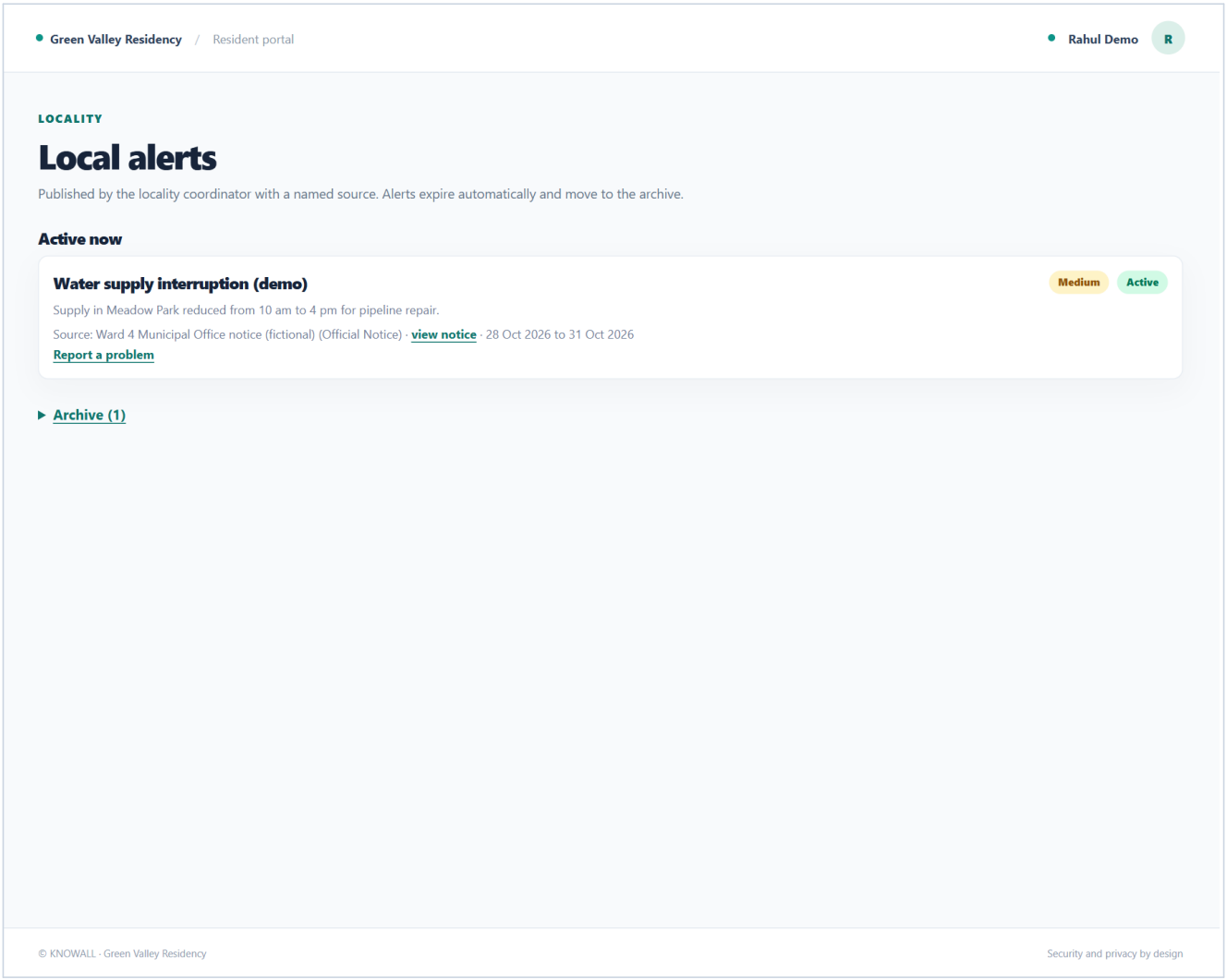


Figure 15 — Local alerts: active alert with severity, source and link; archive of expired alerts; report button.

Society Admin

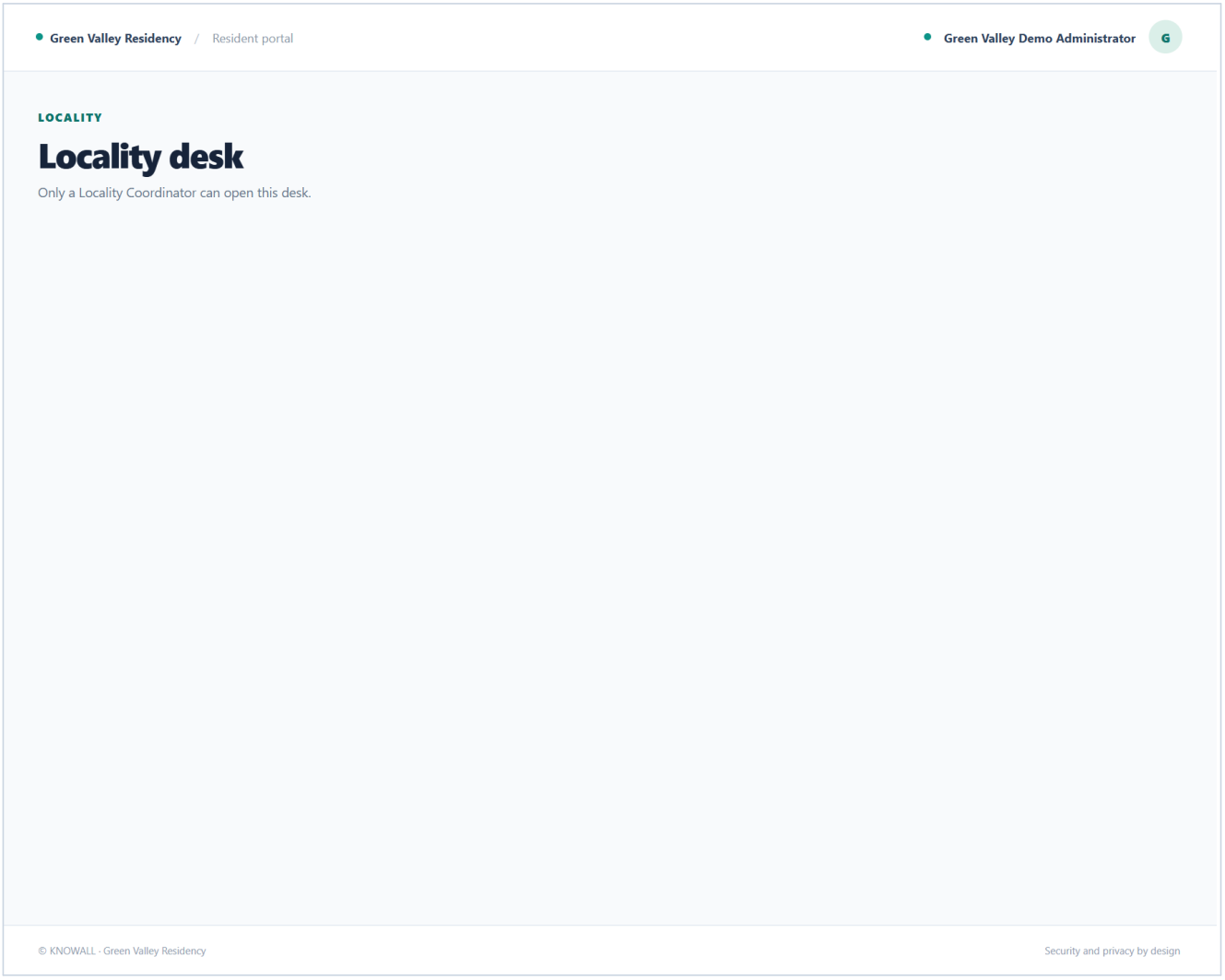


Figure 16 — A Society Admin opening the coordinator desk sees an access message; the API also returns 403.

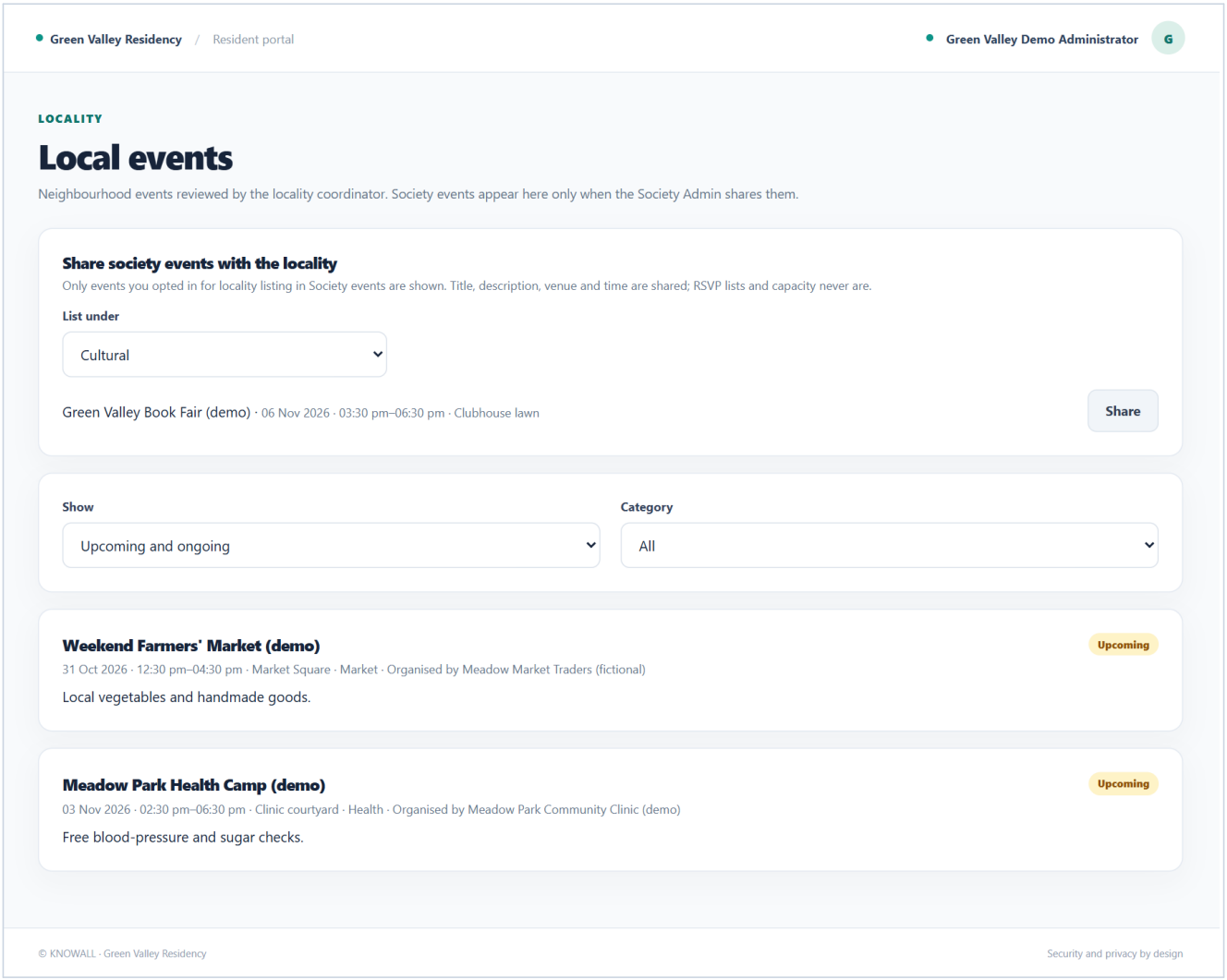


Figure 17 — The Society Admin's share section lists only opted-in, published society events and states that RSVPs and capacity are never shared.

Locality Coordinator

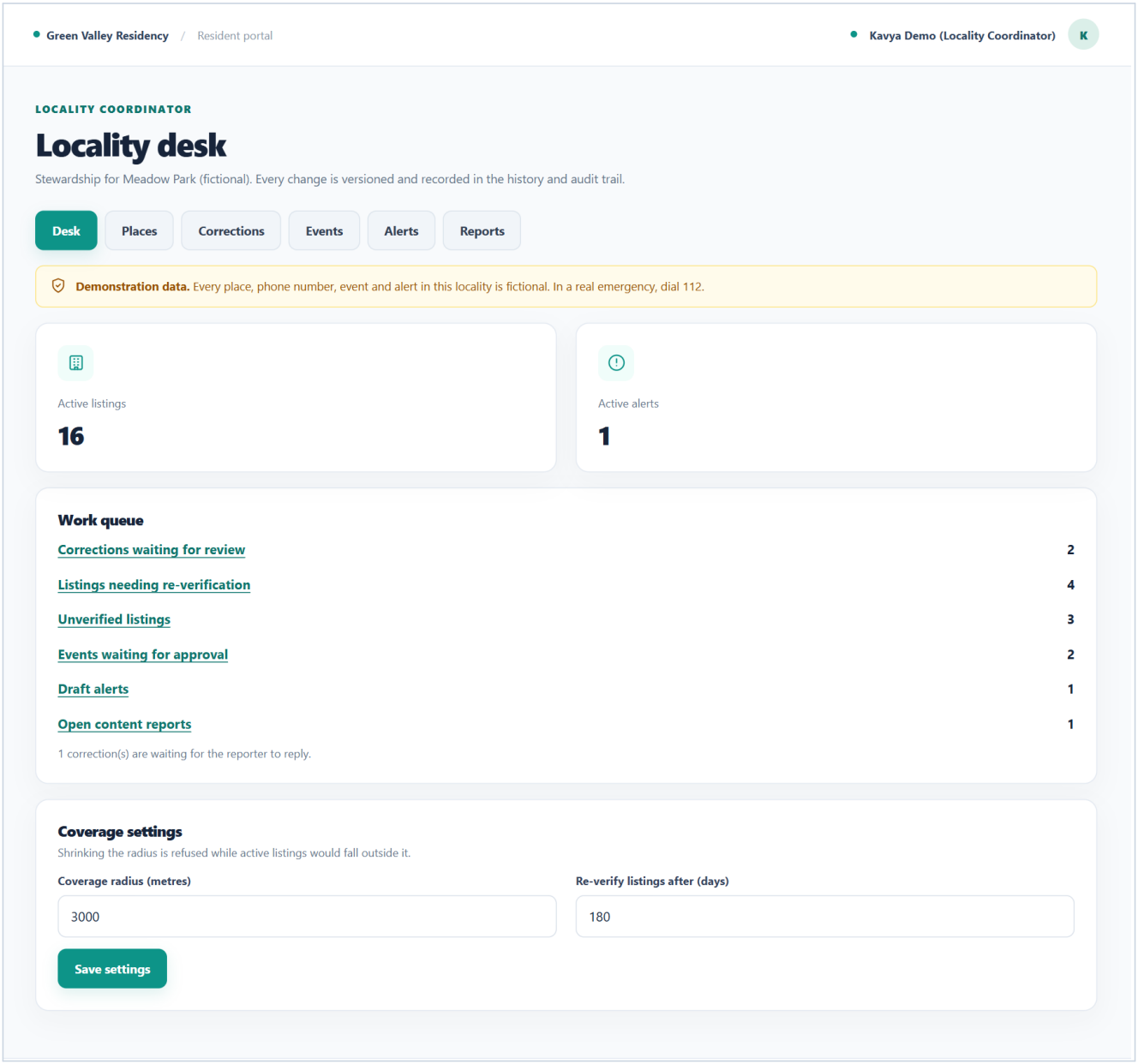


Figure 18 — Locality desk: active listings and alerts, a work queue with counts linking to each queue, and coverage settings.

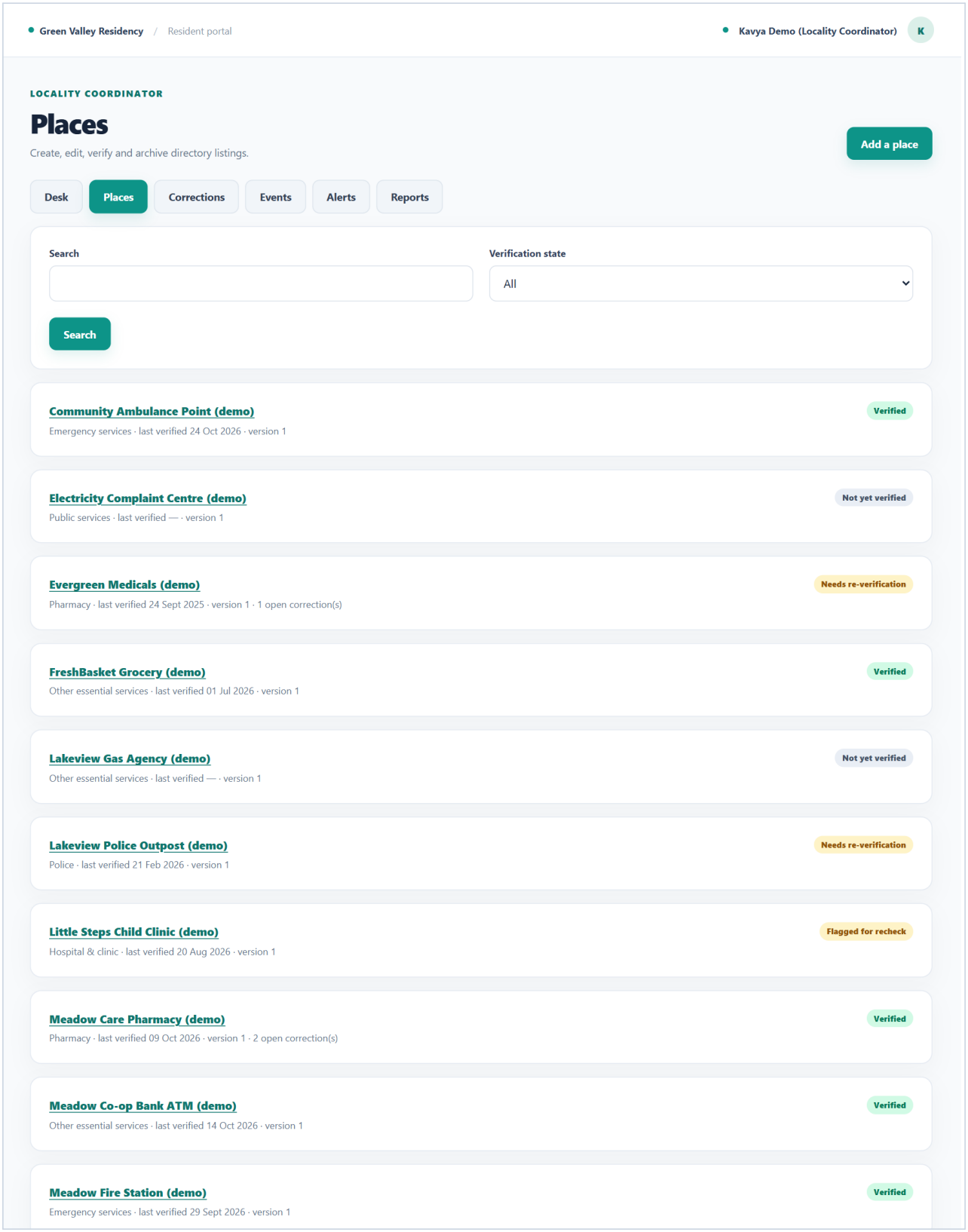


Figure 19 — Places list with verification-state filter, last-verified dates, versions and open-correction counts.

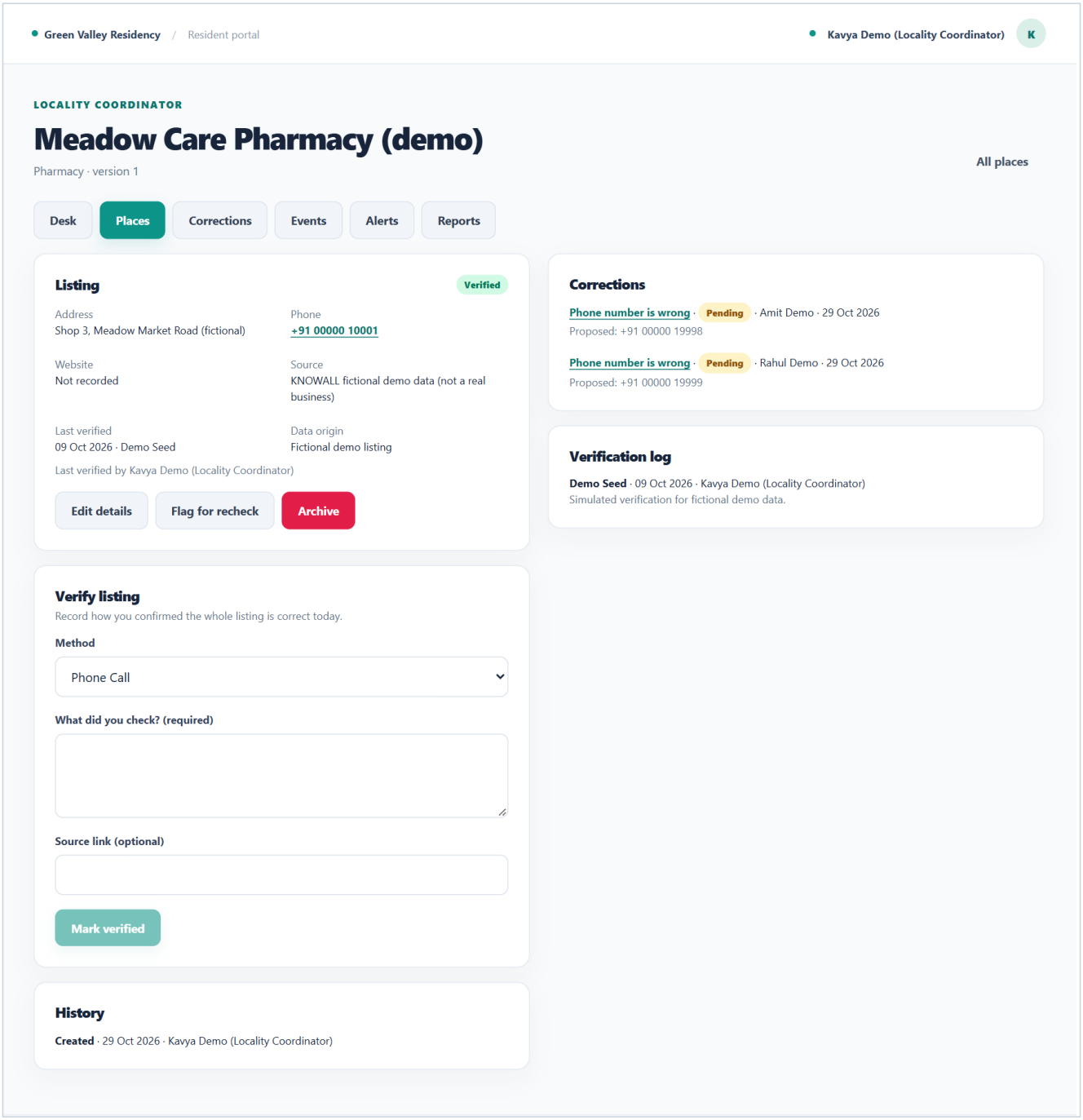


Figure 20 — Place editor: details, flag for recheck, archive/restore with reason, verify form (method + note), correction list, verification log and full history with actors.

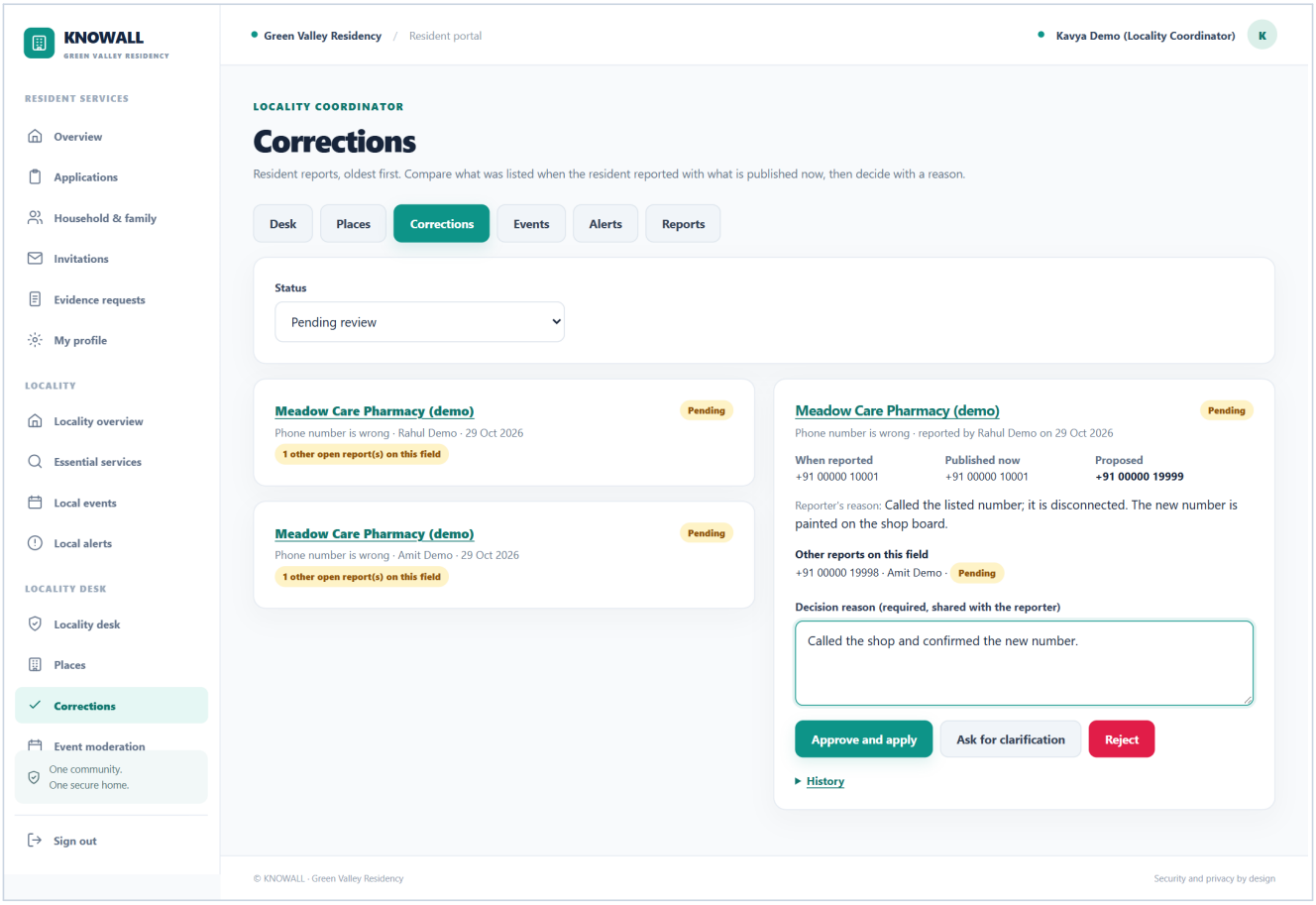


Figure 21 — Correction review: queue with competing-report badges; comparison of value when reported, published now and proposed; mandatory decision reason; approve, ask for clarification or reject.

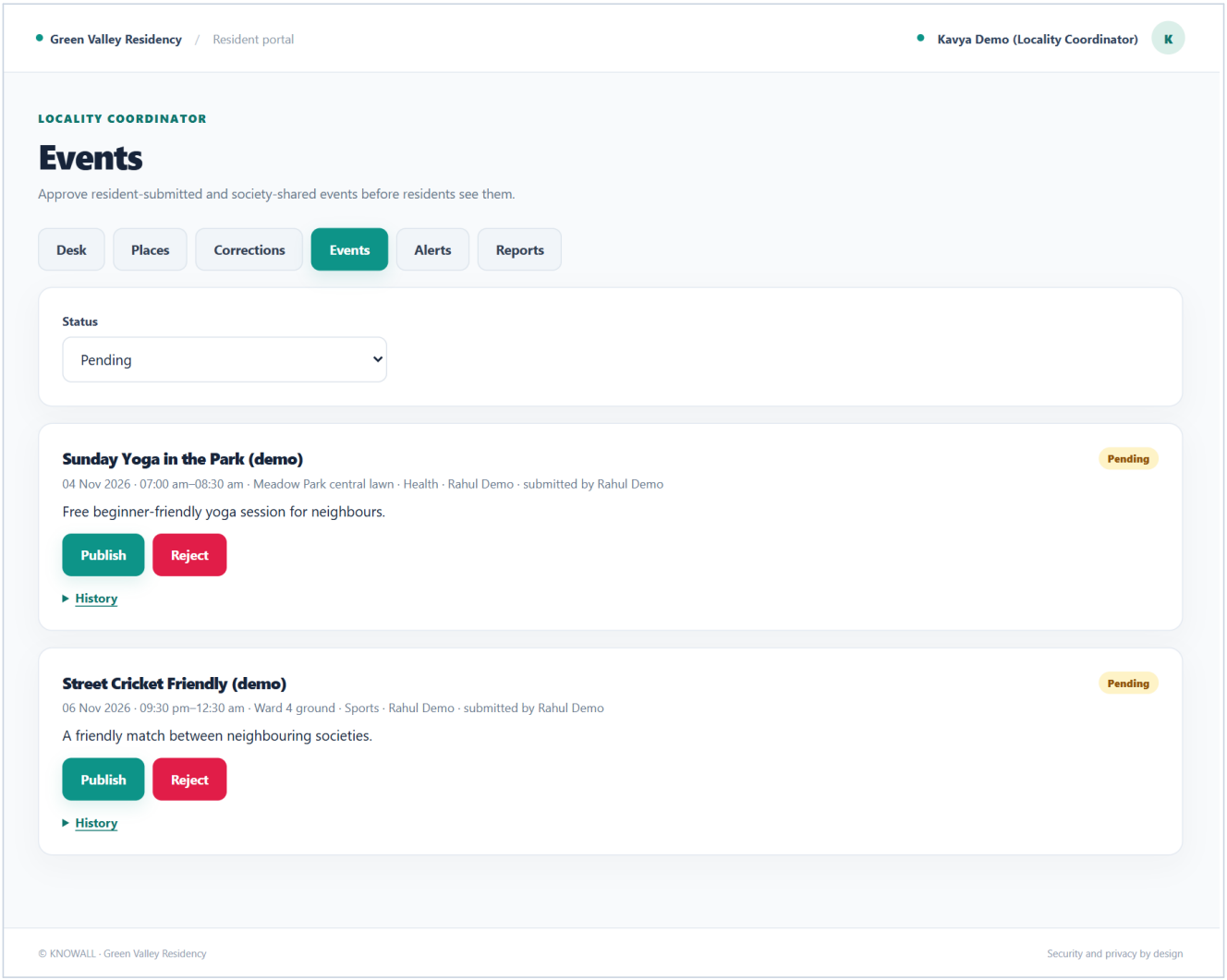


Figure 22 — Event moderation: pending resident suggestion with publish/reject actions requiring a reason.

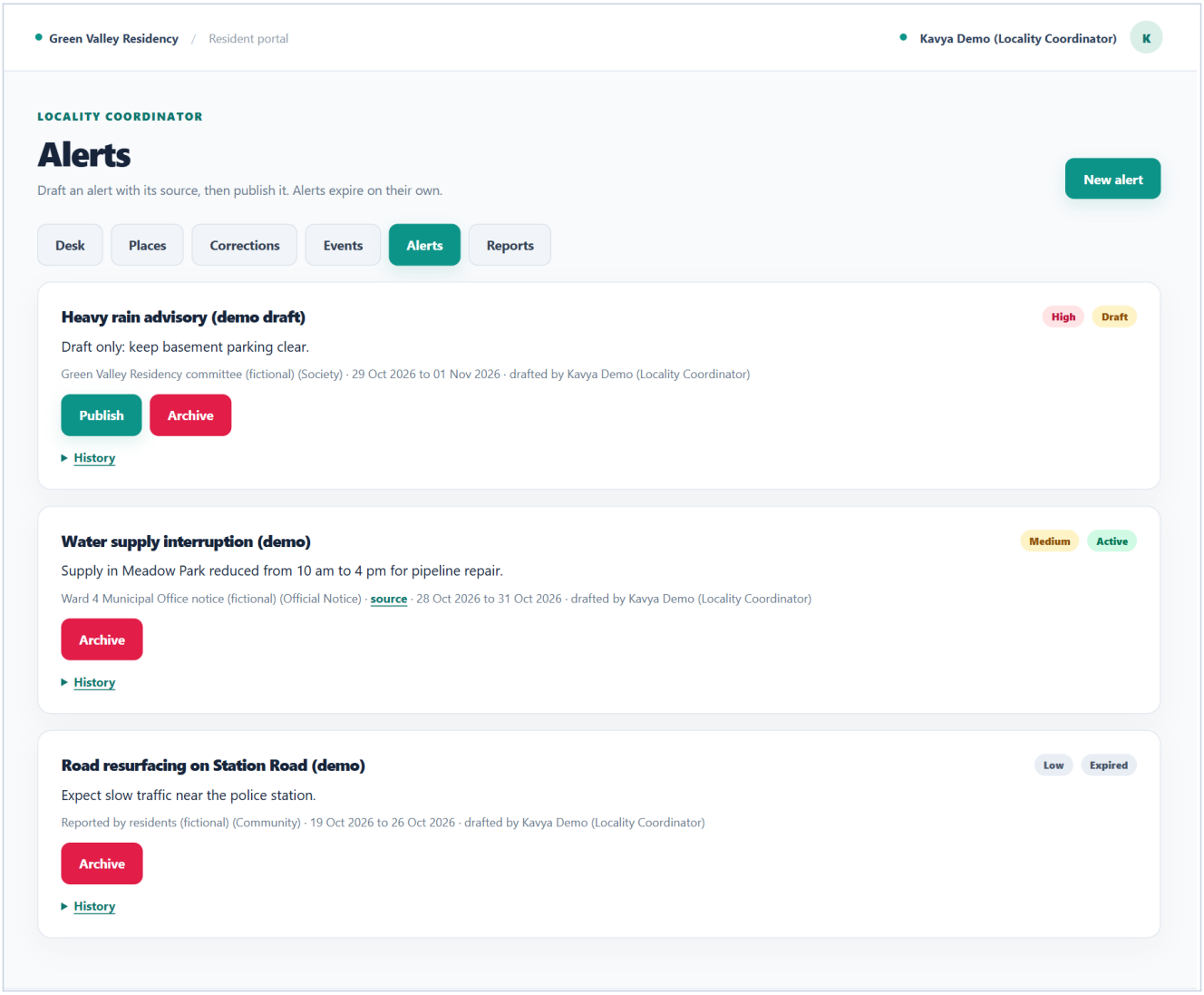


Figure 23 — Alert management: draft and published alerts with source attribution, publish and archive actions, history.

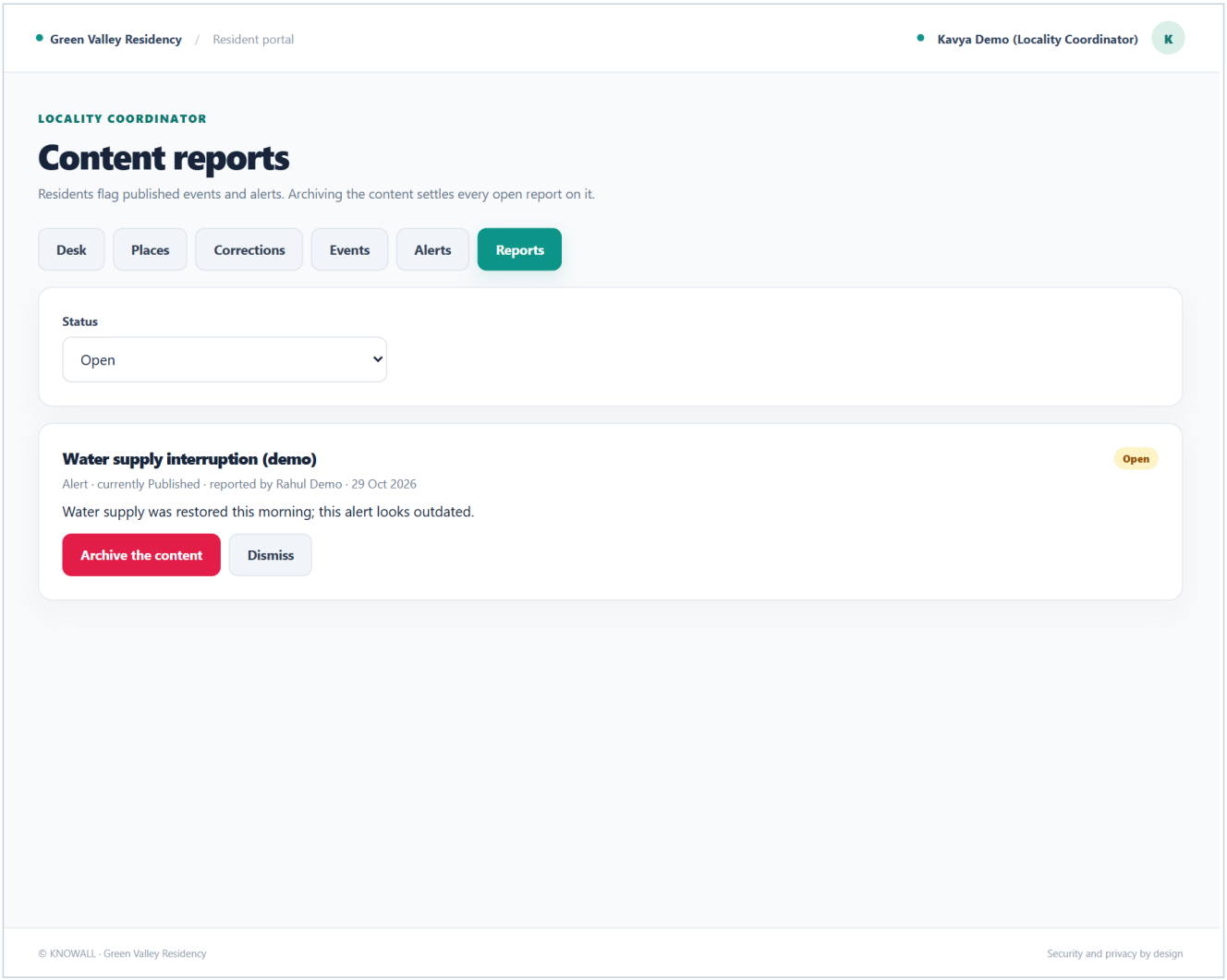


Figure 24 — Content reports: the resident's report on the water-supply alert with archive or dismiss actions.

After approval

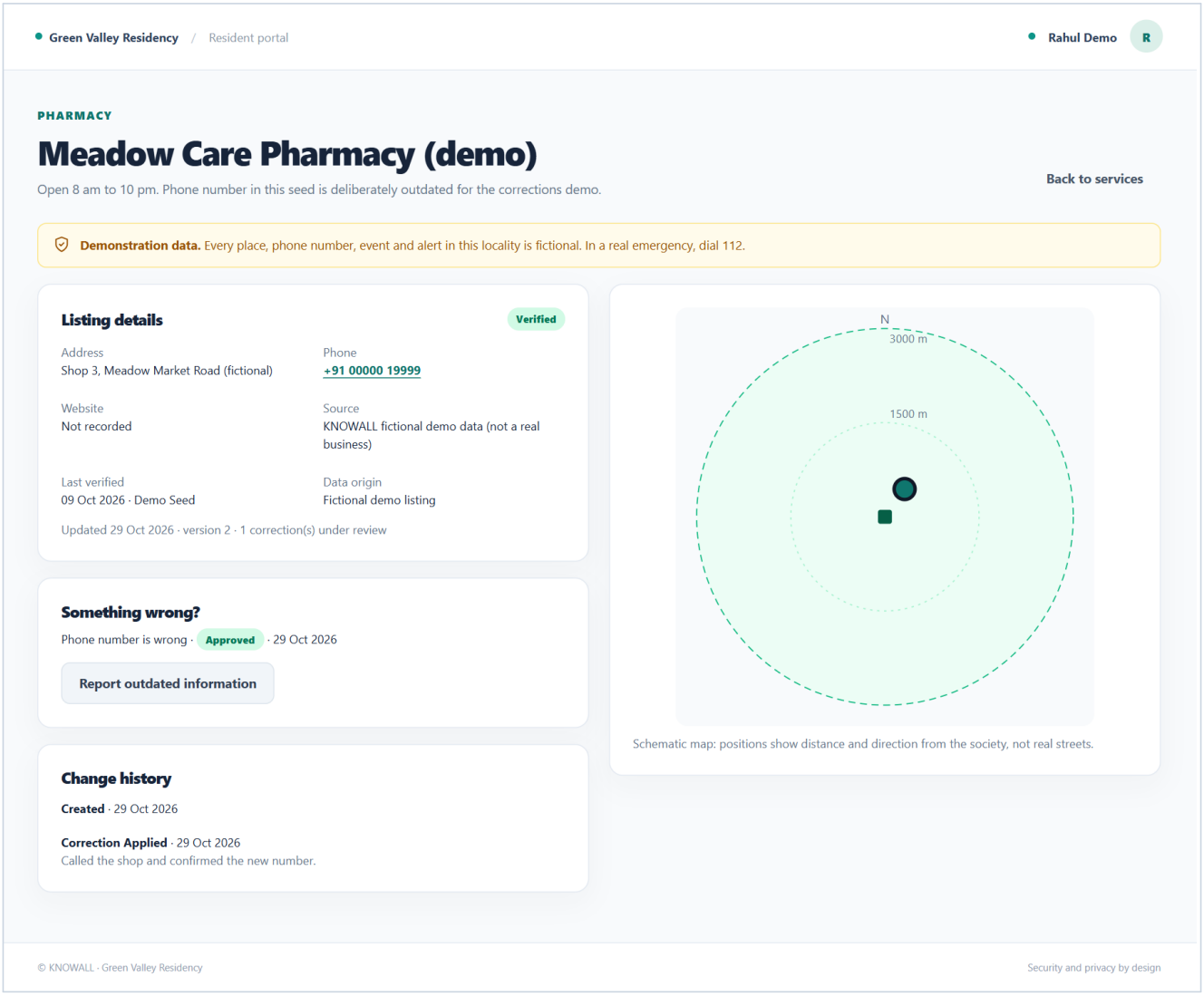


Figure 25 — The pharmacy now shows the corrected number; public history shows "Correction Applied".

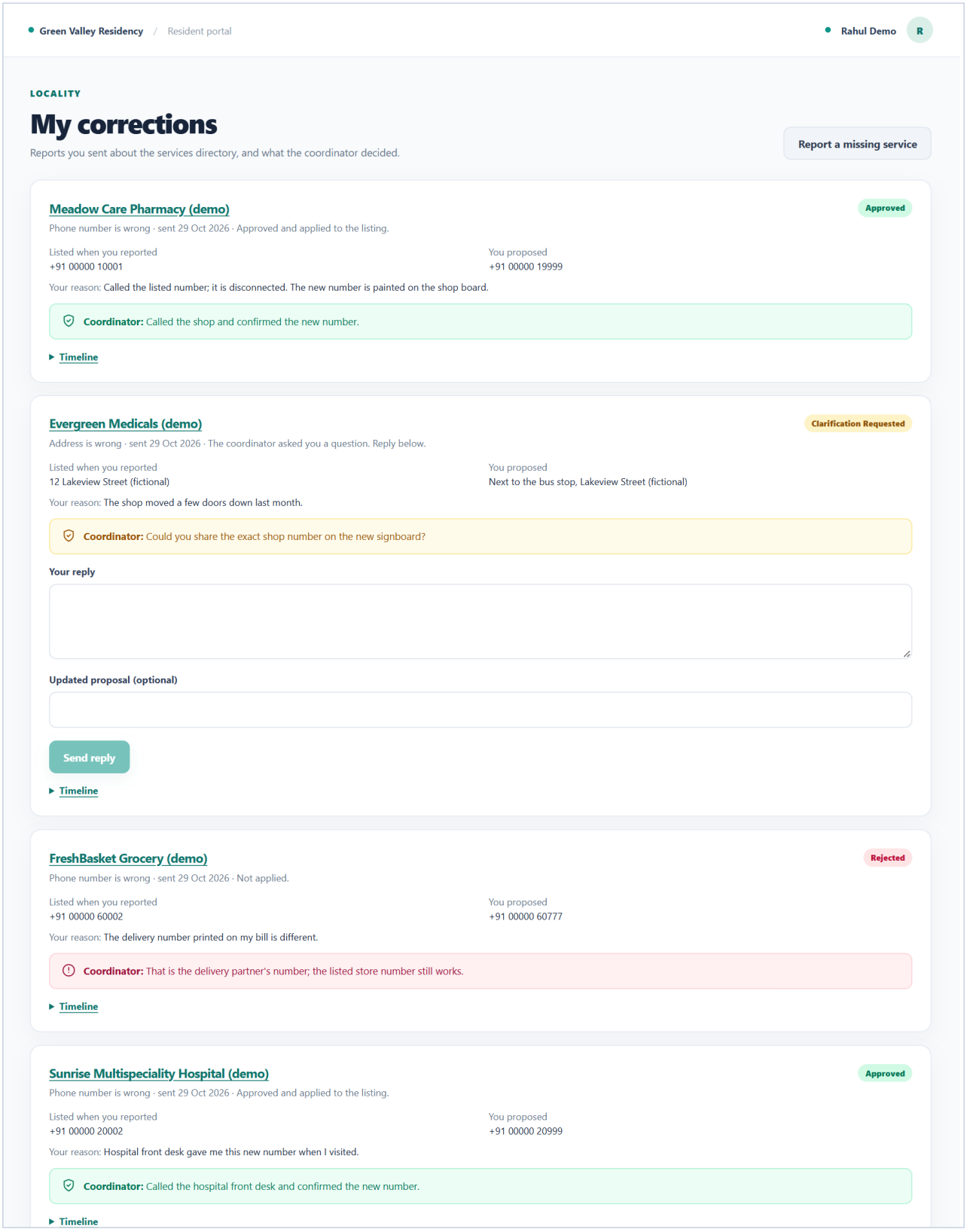


Figure 26 — Rahul sees Approved with the coordinator's reason, and no reviewer name.

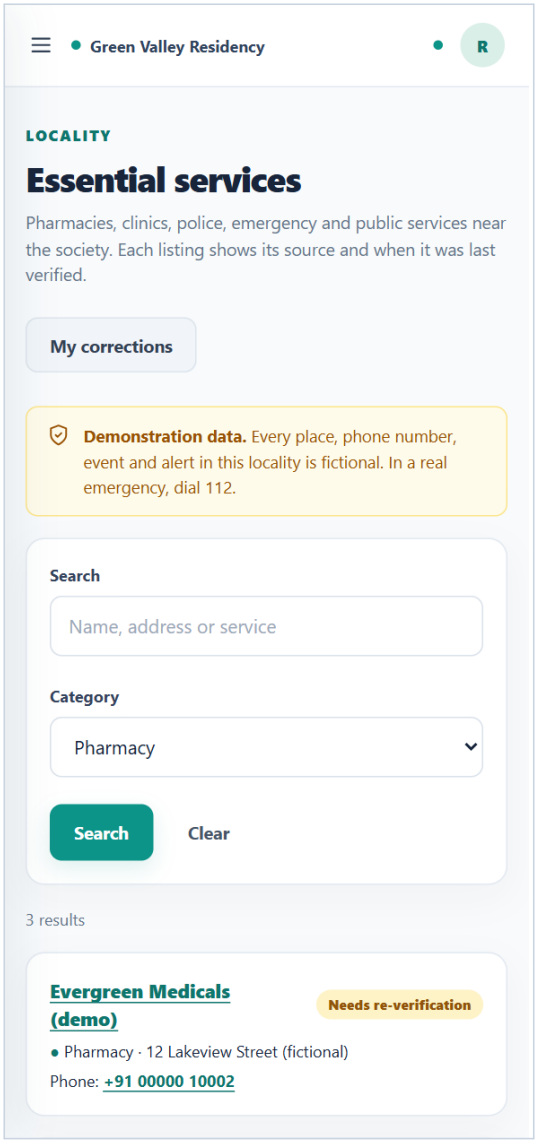


Figure 27 — Mobile layout (390 px wide): the directory remains usable on a phone.

10. API reference

All 37 operations are documented in `backend/app/openapi.json` under the tag "Locality (Module 4)". Mutations require the session cookie and an `X-CSRF-Token` header. Errors use the shared shape `{error: {code, message, fields}}`. Responses are sent with `Cache-Control: no-store`.

Method	Path	Purpose (from OpenAPI)
GET	`/api/locality/profile`	Locality profile, radius coverage and categories with place counts.
GET	`/api/locality/coverage/check`	Is a point inside the coverage radius? Contract for Module 5.
GET	`/api/locality/categories`	Active place categories.
GET	`/api/locality/places`	Search active places (name, address, description, category). Each place has source_label, last_verified_at, verification_state and is_stale.
GET	`/api/locality/places/{place_id}`	Active place detail, my corrections and public change history (no actor identities).
POST	`/api/locality/places/{place_id}/corrections`	Report outdated information. The published value is snapshotted; the listing does not change until a coordinator approves.
POST	`/api/locality/corrections`	Report a missing service (issue_type MISSING_SERVICE, proposed_category required).
GET	`/api/locality/corrections/me`	My corrections with status, decision reason and timeline. Reviewer identities are never included.
POST	`/api/locality/corrections/{correction_id}/respond`	Reply to a clarification request; may update the proposal. Returns the correction to PENDING.
GET	`/api/locality/events`	Published local events (when=upcoming|past|all). Shared society events disappear if Module 3 cancels or un-shares them.
POST	`/api/locality/events/submit`	Submit a local event for review
GET	`/api/locality/events/mine`	Events I submitted, with status and decision reason. Signed-in users.
GET	`/api/locality/events/shareable`	Opted-in, published, not-yet-shared society events (no RSVP or capacity data).
GET	`/api/locality/alerts`	Published alerts grouped into active, scheduled and past (expired or archived).
GET	`/api/locality/feed`	Curated feed: active alerts pinned by severity, then events and directory updates.
POST	`/api/locality/reports`	Report a published event or alert.
GET	`/api/locality/admin/summary`	Work-queue counts for the coordinator desk.
PATCH	`/api/locality/admin/profile`	Update locality name, description, centre, radius or stale_after_days.
GET	`/api/locality/admin/places`	All places including archived; filter state=VERIFIED|STALE|UNVERIFIED|NEEDS_RECHECK|ARCHIVED|ACTIVE.
POST	`/api/locality/admin/places`	Create a place (starts UNVERIFIED). REAL_WORLD listings need source_url.
GET	`/api/locality/admin/places/{place_id}`	Place with verification log, corrections and full history (with actors).
PATCH	`/api/locality/admin/places/{place_id}`	Edit a place. expected_version is required; a stale version returns 409 PLACE_VERSION_CONFLICT.
POST	`/api/locality/admin/places/{place_id}/verify`	Record a whole-listing verification (PHONE_CALL, SITE_VISIT, OFFICIAL_SOURCE needs source_url, DOCUMENT).

Method	Path	Purpose (from OpenAPI)
POST	`/api/locality/admin/places/{place_id}/mark-stale`	Flag a place for recheck.
POST	`/api/locality/admin/places/{place_id}/archive`	Archive (hide from residents).
POST	`/api/locality/admin/places/{place_id}/restore`	Restore an archived place.
GET	`/api/locality/admin/corrections`	Correction queue, oldest first, with published_value and competing_open_corrections.
GET	`/api/locality/admin/corrections/{correction_id}`	Correction with history, related corrections and the current listing.
POST	`/api/locality/admin/corrections/{correction_id}/decision`	APPROVE (applies the field and records a field-scoped CORRECTION_REVIEW verification), REJECT, or REQUEST_CLARIFICATION. A reason is mandatory. Coordinators cannot decide their own corrections.
GET	`/api/locality/admin/events`	All local events with submitter and history.
POST	`/api/locality/admin/events/{event_id}/decision`	PUBLISH, REJECT (from PENDING) or ARCHIVE (from PUBLISHED) with a reason.
GET	`/api/locality/admin/alerts`	All alerts including drafts, with history.
POST	`/api/locality/admin/alerts`	Create a DRAFT alert. OFFICIAL_NOTICE needs source_url; at most 30 days; must expire in the future.
POST	`/api/locality/admin/alerts/{alert_id}/publish`	Publish a draft alert.
POST	`/api/locality/admin/alerts/{alert_id}/archive`	Archive an alert with a reason.
GET	`/api/locality/admin/reports`	Content reports on events and alerts.
POST	`/api/locality/admin/reports/{report_id}/decision`	DISMISS, or ARCHIVE_CONTENT (archives the target and settles every open report on it).

11. Validation, security and privacy

Concern	How it is handled
Authorization	Every route uses Module 1's requires_permission; the UI only hides what Flask already refuses. Society Admin and Platform Admin get 403 on management routes.
CSRF	All mutations require the X-CSRF-Token header (verified in tests and the HTTP demo).
Mass assignment	Each endpoint accepts an explicit whitelist of JSON keys; unknown keys (e.g. status) return 400.
Unsafe links	Only absolute http(s) URLs are stored; the UI renders other values as plain text and opens links with rel=noopener noreferrer nofollow.
Search injection	Parameterised queries; LIKE wildcards escaped; search text limited to 100 characters.
Data scope	Locality resolved server-side from the configured society; ids from another locality return 404.
Privacy	Reporter identities hidden from other residents; reviewer identities hidden from reporters; RSVP, capacity and volunteer data from Module 3 never copied or returned.
Integrity	Database check constraints (coordinate pairs, ranges, verified-date, time windows, official source URL), optimistic version check, snapshot comparison and row locks.
Accountability	Every create, edit, verification, decision, publication and archive writes locality_history and a LOCALITY_* audit log entry with actor and reason.
Abuse limits	One open report per field per resident, 10 open reports per resident, one open content report per item per resident.
Honest data	Seed data is labelled fictional everywhere; seeded verifications use the DEMO_SEED method; emergency listings remind users to dial 112.

12. Testing and verification

All results below were produced by actually running the commands during development on 29 October 2026.

Check	Command	Result
Full backend suite	python -m pytest -q	250 passed, 0 failed
Module 4 backend tests	pytest tests/test_locality_api.py	15 passed
Full frontend suite	node --test tests/*.test.cjs	92 passed, 0 failed
Module 4 frontend tests	node --test tests/locality.test.cjs	12 passed
Type check	npx tsc --noEmit	No errors
Lint	npx eslint (Module 4 files)	No warnings
Production build	npx next build	Compiled; all 13 locality routes generated
Migration on PostgreSQL	flask db upgrade / db check	Applied; no model–schema drift
Seed repeat-safety	seed_locality twice	16 created, then 0 created
HTTP end-to-end demo	scripted resident → coordinator flow	15/15 steps passed

What the backend tests cover

- Directory browsing, search, % escaping, category filter, pagination validation, archived listings hidden, coverage check.
- Role boundaries for unauthenticated, applicant, resident, Society Admin, Platform Admin and coordinator.
- Place lifecycle: validation (javascript: URL, outside coverage, bad phone, unknown category, REAL_WORLD without source, unknown keys), duplicate detection, version conflict, verify rules, mark stale, archive, restore, history and audit entries.
- Coverage shrink protection on the locality profile.
- Main demo: report, duplicate prevention, queue comparison, reason required, approve, listing updated, reporter sees reason without reviewer identity, field-scoped verification and audit entry.
- Rejection and clarification round-trip; only the reporter can reply.
- Stale-snapshot conflict, self-review refusal, MOVED_OR_CLOSED archiving, missing-service report.
- Event suggestion and approval; society-event sharing requiring opt-in, no RSVP leakage, disappearance after Module 3 cancellation.
- Alert lifecycle and feed; content report archiving; CSRF enforcement.

SQLite is used for the automated tests, where FOR UPDATE is a no-op, so the tests prove the rules rather than concurrency; the migration and seed were additionally verified on PostgreSQL 18.

13. Seed data and demo accounts

`python -m scripts.seed_locality` is repeat-safe: it never deletes, resets or reactivates anything. It requires `scripts.seed_green_valley` to have run first.

Account	Role	Use in the demo
kavya@greenvalley.test	Locality Coordinator (no household)	Reviews corrections, moderates events and alerts, maintains places
rahul@greenvalley.test	Resident (owner, B-402)	Finds the pharmacy, reports the outdated phone
admin@greenvalley.test	Society Admin	Reads the locality and shares opted-in society events; cannot moderate

Passwords come from environment variables in the untracked backend/.env (KNOWALL_SEED_COORDINATOR_PASSWORD is new) and are never printed by the seed scripts.

Verification state	Seeded places (all fictional, labelled "(demo)")
Verified	Meadow Care Pharmacy (phone deliberately outdated for the demo), Sunrise Multispeciality Hospital, Community Ambulance Point, Meadow Park Community Clinic, Meadow Park Police Station, Meadow Fire Station, Meadow Park Post Office, Meadow Co-op Bank ATM, FreshBasket Grocery
Needs re-verification (stale)	Evergreen Medicals, Lakeview Police Outpost, Ward 4 Municipal Office
Not yet verified	NightOwl 24x7 Chemist, Lakeview Gas Agency, Electricity Complaint Centre (also has no map location)
Flagged for recheck	Little Steps Child Clinic

Also seeded: two published upcoming events and one past event, one pending resident event; one active official-notice alert, one expired community alert and one draft alert.

14. How to run Module 4

From `backend/` with the virtual environment active (Windows PowerShell):

```
# backend/.env must contain KNOWALL_SEED_COORDINATOR_PASSWORD (12+ chars, different from the others)
python -m flask --app app:create_app db upgrade   # applies e5c1a9d3f742
python -m flask --app app:create_app db current   # expect e5c1a9d3f742 (head)
python -m scripts.seed_green_valley
python -m scripts.seed_locality
python -m flask --app app:create_app run --debug   # port 5000

cd ..\frontend
npm run dev   # http://localhost:3000 → sign in → Locality
```

No new Python or npm package is required. Restart Flask after upgrading: a server started before the locality routes existed answers 404 for `/api/locality/*`.

Demo script (summary)

- Sign in as Rahul → Locality → Essential services → search “pharmacy” → open Meadow Care Pharmacy.
- Report outdated information → Phone number is wrong → new number + reason → Submit. The listing is unchanged; My corrections shows Pending.
- Sign in as admin → the desk is refused (403).
- Sign in as Kavya → lands on the desk → Corrections → compare values → enter a reason → Approve and apply.
- Back as Rahul → the pharmacy shows the new number, history shows Correction Applied, My corrections shows Approved with the reason; the feed shows “Listing corrected”.
- Optional: reject path, clarification path, conflict after a direct edit, event sharing, alert report and archive (full script in docs/module-4/README.md §7).

15. Limitations and future work

Limitation	Reason	Possible next step
Schematic map instead of street tiles	The locality is fictional; real tiles would place invented businesses on real streets.	Add a tile map once real, sourced REAL_WORLD listings exist.
Seed centre coordinates are a placeholder	Arbitrary point, not a survey of any real neighbourhood.	Configure the real centre via desk settings.
No geocoding	Coordinates are entered by hand.	Integrate a geocoder with source attribution.
No notifications	No platform notification contract yet (same as Modules 2–3).	Notify reporters on decisions once the contract exists.
One locality per society	Matches Module 1's single-society design.	Allow multiple coverage areas.
Unranked ILIKE search	Simple and portable.	PostgreSQL full-text search with ranking.
Concurrency proven by design, not load-tested	Unit tests use SQLite.	Concurrent-decision test on PostgreSQL.

16. Files added and changed

Area	Files
Backend (new)	app/modules/locality/__init__.py, common.py, models.py, directory.py, corrections.py, content.py; migrations/versions/e5c1a9d3f742_module4_locality.py; scripts/seed_locality.py; tests/test_locality_api.py
Backend (small additive edits)	app/__init__.py (register blueprint), identity/access/permissions.py (locality.read, coordinator permissions), identity/models/role.py (new role, length 30), app/openapi.json (locality paths), .env.example
Frontend (new)	lib/locality-types.ts, lib/locality-api.ts; features/locality/{shared, overview, directory, place-detail, correction-form, my-corrections, events, alerts}.tsx; features/locality/manage/{common, desk, places, place-form, place-editor, corrections, events, alerts, reports}.tsx; 13 route files under app/locality/; tests/locality.test.cjs
Frontend (small edits)	components/shell.tsx (locality nav groups, active-link rule), lib/login-routing.ts (coordinator landing page)
Documentation	docs/module-4/README.md; pointer in the main README.md
Not changed	No Module 2 or Module 3 source file was modified; no existing table or column was removed.